Apostila: Cluster Pequeno com Slurm, GPU e Infraestrutura como Código

Público: iniciantes em gerenciamento de recursos computacionais e em Infraestrutura como Código (IaC), com experiência básica em programação.

Objetivo: ensinar a montar, de forma automatizada e reproduzível, um cluster pequeno de computação em que as tarefas são enfileiradas e executadas em máquinas com GPU.

Ambiente de referência: Linux baseado em Ubuntu 24.04, incluindo distribuições derivadas (como o Linux Mint 22), com acesso `sudo` e, opcionalmente, uma GPU NVIDIA. Sem GPU, todas as partes continuam válidas, exceto as que dependem de acelerador.

Princípio de leitura: esta apostila ensina conceitos antes de comandos. Nomes de campos, pacotes e opções mudam entre versões. Quando isso acontecer, o conceito continua válido, e a própria ferramenta (`--help`, `man`, `ansible-doc`) indica a forma atual. Os exemplos de saída são ilustrativos, e os valores da sua máquina serão diferentes.

Sumário

Parte 1: Introdução

- **1.1** O que é um cluster de computação e por que usar um
- **1.2** Sistema operacional Linux e o gerenciador de pacotes apt
- **1.3** GPU, driver e ferramentas de estado
- **1.4** O que é Infraestrutura como Código (IaC)
- **1.5** O que é Ansible
- **1.6** O que são YAML e Jinja2
- **1.7** O que é Slurm
- **1.8** O que é munge
- **1.9** O que são GRES e cgroups
- **1.10** O que é Multipass
- **1.11** O que são MariaDB e slurmdbd
- **1.12** O que é NAS e o protocolo NFS
- **1.13** Como as peças se encaixam

Parte 2: Preparando o ambiente

- **2.1** Verificando o acelerador (GPU)
- **2.2** Levantando os dados de hardware
- **2.3** Instalando o Ansible
- **2.4** Criando a estrutura do projeto

Parte 3: Ansible na prática

- **3.1** Configuração do Ansible (`ansible.cfg`)
- **3.2** Inventário: quais máquinas existem
- **3.3** Variáveis: a fonte única da verdade
- **3.4** Playbook: a receita de configuração
- **3.5** Tarefas e módulos
- **3.6** Idempotência: executar de novo sem efeito colateral

Parte 4: Pacotes e autenticação

- **4.1** Instalação de pacotes (role `base`)
- **4.2** Diretórios de estado e permissões
- **4.3** munge: chave e serviço (role `munge`)
- **4.4** Testando a autenticação com `munge -n | unmunge`

Parte 5: Slurm, componente por componente

- **5.1** Arquitetura: `slurmctld`, `slurmd`, `slurmdbd` e os comandos do usuário
- **5.2** `slurm.conf`: o arquivo principal de configuração
- **5.3** Nós (`NodeName`) e partições (`PartitionName`)
- **5.4** Escalonador e seleção de recursos

Parte 6: GPU como recurso agendável

- **6.1** GRES: declarando GPUs (`gres.conf`)
- **6.2** Pedindo GPUs em um job (`--gres=gpu:1`)
- **6.3** cgroups e isolamento de dispositivos (`cgroup.conf`)

Parte 7: Templates: gerando configuração a partir de variáveis

- **7.1** Jinja2: variáveis, laços e condicionais
- **7.2** `slurm.conf.j2` passo a passo
- **7.3** `gres.conf.j2` e a diferença entre nó com GPU física e nó sem GPU física
- **7.4** Por que nunca editar arquivos gerados

Parte 8: Integração do playbook ao cluster funcionando

- **8.1** Como o playbook percorre as roles
- **8.2** Handlers: reiniciar serviços só quando a configuração muda
- **8.3** Ordem de execução e dependências entre serviços
- **8.4** Fluxo completo de uma mudança de configuração

Parte 9: Validação do cluster

- **9.1** `sinfo`, `scontrol show node` e `squeue`
- **9.2** Primeiro job: `srun` e `sbatch`

- **9.3** Verificando a GPU dentro de um job

Parte 10: Expansão

- **10.1** Nós simulados no mesmo host e o diretório de spool
- **10.2** Múltiplas máquinas virtuais e distribuição da chave munge
- **10.3** Accounting com slurmdbd e MariaDB

Parte 11: Solução de problemas

- **11.1** Tabela de sintomas e causas
- **11.2** Como ler logs (`journalctl` e logs do Slurm)
- **11.3** Hábitos de depuração

Parte 12: Conclusão

- **12.1** O que foi construído
- **12.2** Boas práticas para reutilizar a automação em outras máquinas
- **12.3** Próximos passos

Apêndices

- **A.** Glossário rápido
- **B.** Comandos de referência em uma página
- **C.** Referências para aprofundamento (fontes públicas e científicas)

Parte 1: Introdução

Esta parte apresenta, uma por vez, as tecnologias que compõem um cluster pequeno automatizado. Cada item traz a definição, o problema que resolve e um exemplo curto. Os exemplos ilustram o conceito. As partes seguintes usam os mesmos conceitos na prática.

1.1 O que é um cluster de computação e por que usar um

Um **cluster** é um conjunto de máquinas conectadas em rede que funcionam como um sistema único para os usuários. Em vez de cada pessoa usar uma máquina quando ela está livre, os usuários enviam **tarefas** (chamadas jobs) para uma fila, e um programa decide em qual máquina e com quais recursos cada tarefa roda.

Quando o cluster é usado para cálculo intensivo, como treino de modelos de aprendizado de máquina, a área se chama **HPC** (High Performance Computing, computação de alto desempenho). Os problemas que um cluster resolve são de três tipos:

- **Ociosidade:** equipamentos caros ficam parados enquanto outros esperam por eles.
- **Falta de ordem:** sem fila, quem chega primeiro ocupa o recurso, e os demais esperam sem previsão.
- **Falta de reprodutibilidade:** a configuração de cada máquina é feita à mão, e ninguém lembra exatamente o que foi feito.

Os dois primeiros problemas são tratados pelo gerenciador de recursos, o Slurm. O terceiro é tratado pela Infraestrutura como Código, com o Ansible.

1.2 Sistema operacional Linux e o gerenciador de pacotes apt

Linux é um sistema operacional. **Ubuntu** é uma distribuição Linux, ou seja, uma versão do Linux com um conjunto de programas e um modo próprio de instalar software. Outras distribuições, como o **Linux Mint**, são derivadas do Ubuntu e usam os mesmos pacotes.

Para descobrir qual distribuição e versão está instalada:

```
cat /etc/os-release
```

Em sistemas Debian e Ubuntu, os programas são instalados pelo **apt**, o gerenciador de pacotes, a partir de repositórios oficiais:

```
sudo apt update                 # atualiza a lista de pacotes disponíveis
sudo apt install slurm-wlm      # instala um pacote
apt policy slurm-wlm            # mostra a versão disponível e a instalada
```

O prefixo `sudo` concede privilégio de administrador. Instalar serviços e editar arquivos em `/etc/` exige esse privilégio, e isso aparece com frequência ao longo da apostila.

1.3 GPU, driver e ferramentas de estado

A **GPU** (Graphics Processing Unit) é um processador com muitos núcleos pequenos, adequado para executar muitas operações iguais em paralelo. Esse tipo de cálculo é a base do treino de redes neurais, por isso as GPUs são recursos valiosos em um cluster.

O sistema operacional não conversa diretamente com a placa. Quem faz essa ponte é o **driver**, um módulo do kernel Linux. Quando o driver está ativo, a placa aparece como um dispositivo de arquivo, como `/dev/nvidia0` no caso das placas NVIDIA.

Para consultar o estado de uma placa NVIDIA, use a ferramenta de linha de comando:

```
nvidia-smi
```

A saída mostra o modelo da placa, a versão do driver, a versão de CUDA suportada (CUDA é a plataforma de programação usada pelos programas que rodam na GPU) e a memória de vídeo.

Os rótulos da saída mudam entre versões do driver. Guias antigos mostram `Driver Version` e `CUDA Version`, e versões recentes usam outros nomes, como `KMD Version` e `CUDA UMD Version`. O conceito é o mesmo. Não decore os rótulos: leia a saída e procure o conceito.

Para o nome completo do modelo, sem truncamento:

```
nvidia-smi -L
```

Para placas de outros fabricantes, as ferramentas equivalentes são `rocm-smi` (AMD) e `intel_gpu_top` (Intel). A lógica de verificação é a mesma.

1.4 O que é Infraestrutura como Código (IaC)

Infraestrutura como Código é a prática de descrever a configuração de máquinas em arquivos de texto, em vez de configurá-las à mão. Esses arquivos podem ser executados novamente e guardados em um sistema de controle de versão, que registra o que mudou, quando e por quê.

Compare duas formas de instalar um serviço:

Forma manual (difícil de repetir):

```
sudo apt install munge
sudo systemctl enable --now munge
sudo nano /etc/slurm/slurm.conf   # e edita à mão, lembrando de cada linha
```

Forma IaC (descrita em arquivo):

```
- name: Instalar munge
  ansible.builtin.apt:
    name: munge
    state: present

- name: Iniciar munge
  ansible.builtin.systemd:
    name: munge
    state: started
    enabled: true
```

A segunda forma descreve **o que deve existir**, e não **como chegar lá passo a passo**. Se o serviço for removido e o arquivo for executado de novo, o resultado é o mesmo. Essa é a principal vantagem.

Duas propriedades são centrais:

- **Idempotência:** aplicar a mesma configuração várias vezes produz o mesmo estado final.
- **Fonte única da verdade:** a topologia (quantas máquinas, quantas GPUs, quais portas) fica em um lugar só, e os demais arquivos são gerados a partir dele.

  Recomendação: mantenha a pasta do projeto construído nesta apostila sob controle de versão, com uma ferramenta como o Git, e registre uma versão a cada mudança relevante. Se algo quebrar, é possível voltar à configuração que funcionava. O uso do Git não faz parte desta apostila, e nenhum passo depende dele.

Existem várias ferramentas de IaC. Algumas são mais adequadas para **configurar máquinas que já existem** (como o Ansible, o Puppet, o Chef e o SaltStack). Outras são mais adequadas para **criar infraestrutura em nuvem**, como o Terraform e o OpenTofu. Esta apostila usa o Ansible, que atende ao caso de máquinas físicas ou virtuais já disponíveis.

1.5 O que é Ansible

O **Ansible** executa tarefas de configuração em máquinas a partir de arquivos YAML. Ele não exige programa instalado nas máquinas gerenciadas: usa SSH para acessá-las, ou execução local quando o alvo é a própria máquina.

Os quatro conceitos centrais:

Conceito	O que é	Exemplo de arquivo
Inventário	Lista das máquinas e seus grupos	`inventory/hosts.yml`
Variáveis	Valores usados pelas tarefas e pelos templates	`group_vars/all.yml` e variáveis de host no inventário
Playbook	Lista ordenada de roles ou tarefas, com os hosts-alvo	`site.yml`
Role	Pasta com tarefas, handlers e templates de um assunto	`roles/slurm/`

Teste de conexão com o modo ad hoc (um comando único, sem arquivo de playbook). O padrão `all` aponta para todos os hosts do inventário:

```
ansible all -m ping
```

Se responder `pong`, o Ansible consegue operar na máquina. Aqui, `ping` é um módulo do Ansible, não o comando de rede.

Inventário mínimo para executar tudo na própria máquina. O nome do host é o nome curto da máquina, que `hostname -s` retorna:

```
all:
  children:
    controller:
      hosts:
        <hostname>:
          ansible_connection: local
```

`ansible_connection: local` diz ao Ansible para executar na própria máquina, sem SSH. Isso permite testar a automação completa em um único computador antes de aplicá-la a outras máquinas.

1.6 O que são YAML e Jinja2

YAML

YAML é um formato de texto para dados estruturados. A regra mais importante é a **indentação com espaços**. Tabulações não são permitidas.

```
cluster_name: mini-cluster      # chave: valor
nodes:                          # lista (itens começam com traço)
  - name: node01
    cpus: 8
    gpu_count: 1
```

Os traços (`-`) indicam itens de lista. A indentação indica pertencimento: `name`, `cpus` e `gpu_count` pertencem ao item da lista.

Jinja2

Jinja2 é uma linguagem de templates. Um template é um texto com espaços marcados para valores variáveis. O Ansible usa Jinja2 para gerar arquivos de configuração. Há dois tipos de marcação:

- `{{ variavel }}`: insere um valor.
- `{% ... %}`: controla o fluxo, com laços (`for`) e condicionais (`if`).

```
{% for node in nodes %}
NodeName={{ node.name }} CPUs={{ node.cpus }}
{% endfor %}
```

Se a lista `nodes` tiver dois itens, o arquivo gerado terá duas linhas `NodeName=...`. Os nós são declarados uma vez, e o template produz quantas linhas forem necessárias. É esse mecanismo que permite que a configuração cresça sem ser reescrita.

1.7 O que é Slurm

O **Slurm** (Slurm Workload Manager) é um gerenciador de recursos e escalonador de jobs de código aberto, muito usado em supercomputadores e clusters acadêmicos. Ele faz três coisas:

1. **Recebe** jobs dos usuários.
2. **Decide** quando e onde cada job roda, respeitando os recursos pedidos (CPUs, memória, GPUs) e as políticas configuradas.
3. **Executa** o job no nó escolhido e registra o resultado.

Os componentes principais, detalhados na Parte 5:

Componente	Papel
`slurmctld`	Controlador central. Mantém a fila e decide as alocações.
`slurmd`	Daemon de cada nó de computação. Executa os jobs e informa os recursos do nó.
`slurmdbd`	Serviço de histórico (accounting). Registra o que rodou, quem rodou e por quanto tempo.

Comandos que o usuário utiliza:

```
sinfo                          # mostra os nós e seus estados
squeue                         # mostra a fila de jobs
sbatch treino.sh               # envia um script como job, em segundo plano
srun --gres=gpu:1 nvidia-smi # executa um comando diretamente, com GPU alo
scancel 1234                   # cancela o job de número 1234
```

Um script de job (`sbatch`) é um arquivo shell com diretivas `#SBATCH` no início:

```
#!/bin/bash
#SBATCH --job-name=teste-gpu
#SBATCH --gres=gpu:1
#SBATCH --output=saida_%j.log

hostname
nvidia-smi
```

- `--gres=gpu:1` solicita uma GPU.
- `--output=saida_%j.log` grava a saída em arquivo. `%j` é substituído pelo número do job.

Envio e acompanhamento:

```
sbatch teste-gpu.sh          # Submitted batch job 1234
squeue                       # mostra o job na fila ou em execução
cat saida_1234.log           # lê o resultado quando o job termina
```

Slurm é uma entre várias alternativas para gerenciar filas de jobs. Outras opções conhecidas incluem o Open Grid Scheduler (continuação de código aberto do Sun Grid Engine), o PBS/Torque e o LSF, além de soluções baseadas em Kubernetes. A escolha depende do tipo de carga, do suporte a GPUs e da maturidade da comunidade.

1.8 O que é munge

Os componentes do Slurm precisam confiar uns nos outros. Quando o `slurmctld` envia um job ao `slurmd`, o `slurmd` precisa ter certeza de que a mensagem veio de um participante legítimo. Essa autenticação é feita pelo **munge** (MUNGE Uid 'N' Gid Emporium).

Funcionamento, em termos simples:

- Todas as máquinas do cluster compartilham a **mesma chave**, guardada em `/etc/munge/munge.key`.
- O munge usa essa chave para assinar pequenas credenciais trocadas pelos programas.
- Um programa só é aceito pelo outro se a credencial for válida.

Teste isolado, sem o Slurm:

```
munge -n | unmunge
```

Saída esperada (resumida):

```
STATUS:          Success (0)
ENCODE_HOST:     nome-da-maquina
UID:             root
```

`munge -n` cria uma credencial e `unmunge` a valida. Se aparecer `Success`, a autenticação local funciona. O campo `UID` mostra o usuário que executou o

comando: `root` no exemplo, ou o seu próprio usuário quando o comando é executado sem `sudo`.

Quando há várias máquinas, todas precisam ter **exatamente a mesma chave**. Para comparar:

```
sudo md5sum /etc/munge/munge.key
```

Valores diferentes significam que os nós não se autenticam. É um dos erros mais comuns em clusters.

1.9 O que são GRES e cgroups

GRES

Por padrão, o Slurm conhece CPUs e memória. Para recursos adicionais, como GPUs, é preciso declarar um **GRES** (Generic RESource), um recurso genérico com nome livre. Neste tutorial, o recurso é chamado `gpu`.

A declaração acontece em dois arquivos:

1. Em `slurm.conf`, o tipo de recurso é registrado para o cluster inteiro com `GresTypes`, e cada nó informa quantas unidades possui com `Gres`:

```
GresTypes=gpu
NodeName=node01 ... Gres=gpu:1
```

A linha `GresTypes` vem antes das linhas de nó. Sem ela, o Slurm não reconhece o nome `gpu`, e a linha `NodeName` com `Gres=gpu:1` é rejeitada.

1. Em `gres.conf`, o dispositivo físico é apontado:

```
NodeName=node01 Name=gpu File=/dev/nvidia0
```

Com isso, `--gres=gpu:1` passa a ser uma requisição válida no job.

cgroups

cgroups (control groups) é um recurso do kernel Linux que limita e isola o uso de recursos por processo. O Slurm usa cgroups para que cada job veja apenas os recursos que recebeu. Com `ConstrainDevices=yes`, por exemplo, um job que pediu uma GPU não enxerga as outras.

Configuração correspondente, em `cgroup.conf`:

```
CgroupPlugin=autodetect
ConstrainCores=yes
ConstrainRAMSpace=yes
ConstrainDevices=yes
```

Sem cgroups, um job com problema pode consumir a memória de todos os outros. Com cgroups, ele fica confinado aos seus recursos.

1.10 O que é Multipass

Multipass é uma ferramenta da Canonical (a empresa que mantém o Ubuntu) para criar máquinas virtuais Ubuntu rapidamente. Uma máquina virtual é um computador simulado dentro de outro computador.

Ela é útil para simular várias máquinas em rede sem precisar de hardware adicional:

```
multipass launch --name ctl --cpus 2 --memory 2G      # máquina controlado
multipass launch --name node01 --cpus 2 --memory 2G   # máquina de computa
multipass list                                        # mostra as VMs e se
multipass shell ctl                                   # abre um terminal d
```

Máquinas virtuais comuns não têm acesso a GPU física. Elas servem para testar a orquestração e a comunicação entre nós. Testes de desempenho com GPU precisam ser feitos em máquinas com a GPU real.

1.11 O que são MariaDB e slurmdbd

MariaDB é um banco de dados relacional, derivado do MySQL, com licença livre. Ele armazena dados em tabelas.

slurmdbd é o serviço do Slurm que grava o histórico de jobs nesse banco. Comandos de consulta, como `sacct`, leem os dados por meio do slurmdbd.

Fluxo com accounting ativo:

```
sbatch → slurmctld → slurmd (executa o job)
                  ↓
              slurmdbd → MariaDB (guarda o histórico)
                  ↑
            sacct (consulta)
```

Exemplo de consulta:

```
sacct -X --format=JobID,User,Partition,Elapsed,State
```

A consulta mostra, para cada job, quem o enviou, em qual partição, quanto tempo levou e em que estado terminou. Esses dados são a base para analisar o uso do cluster e para políticas de justiça entre usuários (fairshare).

Sem accounting, `sacct` não tem o que mostrar.

1.12 O que é NAS e o protocolo NFS

NAS (Network Attached Storage, armazenamento conectado à rede) é um servidor de arquivos acessado pela rede. Em um cluster, ele permite que todas as máquinas leiam os mesmos dados, sem copiá-los para cada uma.

No Linux, o protocolo mais comum para isso é o **NFS** (Network File System). Uma pasta remota é montada em um caminho local:

```
sudo mount -t nfs servidor:/dados /mnt/datasets
ls /mnt/datasets        # os arquivos aparecem como se fossem locais
```

A versão do protocolo é negociada entre cliente e servidor, e pode ser fixada com a opção `vers=` do comando `mount`, conforme a configuração do servidor. Para que a montagem sobreviva a uma reinicialização, ela é registrada em `/etc/fstab`, ou gerenciada pela automação. Do ponto de vista de um job, o caminho `/mnt/datasets` é uma pasta comum.

Um NAS é uma escolha de armazenamento, não um requisito do Slurm. Ele é útil quando vários nós precisam dos mesmos dados. A escolha da solução (NFS, Ceph, outras) depende do tamanho do cluster, da rede e do tipo de carga.

1.13 Como as peças se encaixam

Fluxo de uma mudança na configuração:

```
Você edita o inventário (por exemplo, adiciona uma máquina)
        │
        ▼
ansible-playbook site.yml
        │
        ├── role base:    instala slurm-wlm e munge
        ├── role munge:   garante o serviço munge em execução
        └── role slurm:   gera slurm.conf, gres.conf e cgroup.conf
                          a partir dos templates Jinja2
                          reinicia slurmctld/slurmd se algo mudou
        │
        ▼
Slurm pronto: sinfo mostra o nó, sbatch executa o job na GPU
```

Você não edita `slurm.conf` diretamente. Você altera a fonte (o inventário ou `group_vars/all.yml`), e o Ansible regenera o arquivo. As próximas partes detalham cada etapa desse fluxo.

Próximo passo

A **Parte 2** começa pelo ambiente: confirmar que a GPU responde (se houver) e levantar os dados da máquina. Nada é instalado antes dessa verificação.

Parte 2: Preparando o ambiente

Objetivo desta parte: confirmar que o sistema reconhece o acelerador (GPU, se houver), levantar os dados da máquina, instalar o Ansible e criar a pasta do projeto que guarda a infraestrutura. Nenhum serviço do Slurm é instalado aqui.

Pré-requisitos: - Linux baseado em Ubuntu 24.04 (ou derivado), com terminal e permissão `sudo`. - Opcional: uma GPU. Sem ela, ignore as verificações que tratam de GPU e faça apenas o levantamento de CPU e memória.

Como ler esta parte

Ferramentas mudam. Rótulos de saída mudam entre versões de driver, nomes de pacote mudam entre versões do sistema, e opções de comando são renomeadas. Por isso, a apostila segue duas regras:

1. **Conceitos antes de comandos.** Antes de cada comando, está explicado o que se quer descobrir. Se o comando mudar, o conceito continua válido.
2. **Use a própria ferramenta para descobrir.** Todo comando aceita `--help`, `man <comando>` ou `ansible-doc <módulo>`. Esses são os caminhos de referência atualizados.

2.1 Verificando o acelerador (GPU)

A verificação de uma GPU tem três camadas. Cada uma responde a uma pergunta diferente:

Camada	Pergunta	Ferramenta típica
Hardware	A placa está presente no barramento da máquina?	`lspci`
Driver	O módulo do kernel que se comunica com a placa está carregado?	`lsmod`
Ferramenta de estado	A placa responde a consultas?	`nvidia-smi` (NVIDIA), `rocm-smi` (AMD)

Esse modelo de três camadas vale para qualquer fabricante. Se a placa mudar, a lógica é a mesma. Só o nome das ferramentas muda.

Camada 1: hardware

```
lspci | grep -iE 'vga|3d|display'
```

A linha lista os dispositivos gráficos no barramento PCI. Você deve ver o nome do fabricante e do modelo. Em máquinas com gráficos híbridos (um integrado e uma placa dedicada), as duas aparecem aqui, e isso é normal.

Se `lspci` não estiver disponível:

```
sudo apt install -y pciutils
```

Camada 2: driver

Para placas NVIDIA:

```
lsmod | grep -i '^nvidia'
```

Para placas AMD, o módulo é `amdgpu`. Se a linha aparecer, o módulo está carregado. Se não aparecer, o driver não está ativo, e a camada 3 deve falhar.

Camada 3: ferramenta de estado

```
nvidia-smi
```

Se a ferramenta responder com uma tabela, a GPU está operacional. Para identificar os campos da saída, consulte o item 1.3 da introdução. Os rótulos mudam entre versões, então leia o conceito e não o texto.

Para o nome completo do modelo, sem truncamento:

```
nvidia-smi -L
```

Para extrair valores específicos de forma legível por máquina:

```
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv
```

Se alguma opção não existir na sua versão, consulte `nvidia-smi --help` e procure a equivalente.

Registro do ambiente

Anote os valores em um arquivo dentro do projeto, para não depender da memória. Isso também permite que qualquer pessoa que reproduza a configuração saiba em qual hardware ela foi testada.

O arquivo `docs/ambiente-host.md` é criado, vazio, pelo script da seção 2.4, junto com a pasta do projeto. Até lá, anote os valores à parte e transfira-os depois para o arquivo, com um registro por máquina no formato abaixo:

```
# Ambiente do host

| Item | Valor |
|---|---|
| Função da máquina | a preencher (ex.: controlador, nó de computação, des
| Hostname curto (`hostname -s`) | a preencher |
```

```
| Distribuição (`/etc/os-release`) | a preencher |
| Tipo de máquina (notebook ou desktop) | a preencher |
| GPU (`nvidia-smi -L`) | a preencher ou "sem GPU" |
| Memória de vídeo | a preencher |
| Versão do driver | a preencher |
| Versão CUDA suportada | a preencher |
| Secure Boot (`mokutil --sb-state`) | a preencher |
| CPUs (`nproc`) | a preencher |
| Memória total (`free -m`) | a preencher |
```

Notas sobre notebook

Se a máquina for um notebook, o limite de consumo da placa aparece na saída do `nvidia-smi` (por exemplo, 80 W). Isso traz consequências para um nó de computação, que um servidor não teria:

- **Suspensão e tampa fechada:** se a máquina suspender, o nó deixa de responder e o Slurm o marca como indisponível.
- **Energia:** rodar cargas pesadas na bateria reduz desempenho e pode desligar a máquina.
- **Temperatura:** cargas longas fazem a GPU reduzir a frequência (thermal throttling), o que afeta medições de desempenho.
- **Gráficos híbridos:** a placa dedicada pode desligar quando não está em uso. Isso é normal e explica variações no tempo de inicialização.

Essas condições não impedem o uso da apostila, mas devem ser registradas, porque afetam a interpretação de resultados.

Secure Boot

O Secure Boot pode impedir o carregamento de módulos do kernel que não estejam assinados, como o driver de algumas GPUs. Verifique o estado:

```
mokutil --sb-state
```

Se o Secure Boot estiver ativo e a camada 2 ou a camada 3 falharem, há duas saídas: desativar o Secure Boot na BIOS/UEFI, ou registrar uma chave de assinatura para o módulo (o gerenciador de drivers do sistema, quando houver, orienta esse processo). Anote a escolha no registro do ambiente.

Se o driver não estiver instalado, o caminho mais direto no Ubuntu e derivados é:

```
ubuntu-drivers devices          # lista os drivers recomendados para a pla
sudo ubuntu-drivers install     # instala o recomendado
sudo reboot
```

Em distribuições com interface gráfica, o gerenciador de drivers do sistema faz a mesma coisa.

> **Referência:** NVIDIA. nvidia-smi: NVIDIA System Management Interface. Guia do usuário, disponível em docs.nvidia.com. Use a

versão do guia correspondente ao driver instalado. **Referência:** Gregg, B. Systems Performance: Enterprise and the Cloud. Addison-Wesley. Capítulos sobre inventário e observação de hardware. (Verifique a edição antes de citar.)

2.2 Levantando os dados de hardware

O Slurm precisa saber quantas CPUs e quanta memória cada nó tem. Esses valores devem ser reais. Se forem maiores que os da máquina, o Slurm marca o nó como indisponível, o que aparece na validação da Parte 9.

Comandos estáveis há muitos anos:

```
hostname -s       # nome curto da máquina
nproc             # número de processadores lógicos disponíveis
free -m           # memória em megabytes
```

Na saída de `free -m`, use o valor da coluna `total`, na linha `Mem:`, e registre no arquivo de ambiente.

Esse valor serve como estimativa inicial, não como valor final. A memória que o Slurm detecta costuma ser um pouco menor que o total do `free -m`, porque parte dela é reservada pelo kernel. Se a memória declarada ao Slurm for maior que a detectada, o nó é marcado como indisponível. Por isso, ao declarar a memória ao Slurm (Parte 3), use um valor com margem, por exemplo 5% abaixo do total. A referência final é o comando `slurmd -C`, apresentado a seguir.

Para a visão completa da CPU:

```
lscpu | head -20
```

Sobre `slurmd -C`

`slurmd -C` mostra a configuração que o Slurm enxerga na máquina. O comando só existe depois da instalação do pacote do Slurm, na Parte 4. Até lá, use `nproc` e `free -m`. Na Parte 4, os valores de `nproc` e `free -m` devem ser comparados com os de `slurmd -C`. Em caso de diferença, prevalece o valor de `slurmd -C`, em especial o campo `RealMemory`.

> **Referência:** Kerrisk, M. The Linux Programming Interface. No Starch Press. Capítulo sobre informações do sistema, que explica a origem desses valores no diretório `/proc`. (Verifique a edição antes de citar.)

2.3 Instalando o Ansible

O Ansible está nos repositórios oficiais do Ubuntu:

```
sudo apt update
sudo apt install -y ansible
```

Confirme a versão:

```
ansible --version
```

Em `ansible --version`, observe duas linhas:

- `ansible [core ...]`: versão do núcleo.
- `config file = ...`: arquivo de configuração em uso. Ele ainda não é o do projeto. Isso muda na Parte 3.

  Dica: se um pacote não for encontrado, use `apt search <termo>` para descobrir o nome correto na sua versão da distribuição. Nomes de pacote mudam com o tempo.

  Referência: Ansible. Installing Ansible. docs.ansible.com. Guia oficial de instalação, com instruções por distribuição.

2.4 Criando a estrutura do projeto

A infraestrutura é mantida em uma pasta que este tutorial chama de `slurm-lab`. Você pode criar a estrutura à mão, mas o script abaixo faz isso de forma repetível. Ele segue a mesma lógica da IaC: o que é criado está descrito em um arquivo.

Crie o arquivo `bootstrap-estrutura.sh` em qualquer pasta, com o conteúdo abaixo:

```
#!/usr/bin/env bash
# Cria a estrutura de pastas e arquivos vazios do projeto slurm-lab.
# Pode ser executado várias vezes: não sobrescreve arquivos existentes.
set -euo pipefail

PROJETO="slurm-lab"

mkdir -p "$PROJETO"/{inventory,group_vars,docs} \
         "$PROJETO"/roles/{base,munge,slurm}/{tasks,handlers} \
         "$PROJETO"/roles/slurm/templates

cd "$PROJETO"

# Cria arquivos vazios apenas se ainda não existirem.
touch -a ansible.cfg inventory/hosts.yml group_vars/all.yml site.yml docs/
for role in base munge slurm; do
  [ -f "roles/$role/tasks/main.yml" ] || echo "[]" > "roles/$role/tasks/ma
done
[ -f roles/slurm/handlers/main.yml ] || echo "[]" > roles/slurm/handlers/m

echo "Estrutura criada em ./$PROJETO"
```

O que o script faz:

- `set -euo pipefail`: interrompe o script na primeira falha, para não deixar a estrutura incompleta. É um padrão de scripts bash seguros.
- `mkdir -p ...`: cria todas as pastas de uma vez. A notação `{a,b}` é uma expansão do bash que gera `a` e `b`.
- `touch -a`: cria o arquivo se não existir, sem alterar o conteúdo se existir.
- `echo "[]" > ...`: cria os arquivos de tarefas como lista vazia. É YAML válido, e permite que o playbook passe na checagem de sintaxe antes de as tarefas serem escritas.

Execute:

```
chmod +x bootstrap-estrutura.sh
./bootstrap-estrutura.sh
cd slurm-lab
```

Confira a estrutura:

```
find . | sort
```

> **Recomendação:** mantenha a pasta `slurm-lab` sob controle de versão, com uma ferramenta como o Git, e registre uma versão a cada mudança relevante. Isso permite voltar a uma configuração que funcionava. O controle de versão não faz parte desta apostila, e nenhum passo depende dele.
>
> **Referência:** Shotts, W. The Linux Command Line. No Starch Press. Para o bash, incluindo `set -euo pipefail` e a expansão de chaves.

Resumo da Parte 2

Ao final, você terá: o acelerador verificado em três camadas (ou a confirmação de que não há GPU), o registro do ambiente em `docs/ambiente-host.md`, o Ansible instalado e a pasta do projeto com a estrutura base.

Os valores de CPU e memória serão confrontados com `slurmd -C` na Parte 4.

Próxima: Parte 3, Ansible na prática.

Parte 3: Ansible na prática

Objetivo desta parte: configurar o Ansible para o projeto e entender os quatro pilares do Ansible (inventário, variáveis, playbook e módulos), aprendendo a verificar cada um antes de aplicar qualquer mudança no sistema.

Pré-requisito: Parte 2 concluída, com a pasta do projeto `slurm-lab` criada.

Nesta parte, nada é instalado no sistema. Os exemplos são de leitura, verificação ou execução em pasta temporária. A instalação real começa na Parte 4.

Todos os comandos desta parte são executados a partir da raiz do projeto `slurm-lab`. O motivo é explicado na seção 3.1.

3.1 Configuração do Ansible: `ansible.cfg`

O arquivo `ansible.cfg` ajusta o comportamento do Ansible para um projeto específico. Sem ele, o Ansible usa valores globais, que podem variar de máquina para máquina. Por isso, a configuração fica na própria pasta do projeto e acompanha o projeto.

Ele vem primeiro porque os demais comandos desta parte dependem dele: é o `ansible.cfg` que diz ao Ansible onde estão o inventário e as variáveis do projeto.

```
[defaults]
inventory = inventory/hosts.yml
playbook_dir = .
interpreter_python = auto_silent
retry_files_enabled = False
```

Cada linha:

- **`inventory`:** caminho do inventário, apresentado na seção 3.2. Assim, não é preciso passar a opção `-i` em cada comando.
- **`playbook_dir`:** pasta de referência do projeto. O Ansible carrega automaticamente a pasta `group_vars/` (seção 3.3) quando ela está ao lado do playbook. Comandos avulsos, que não usam playbook, não têm essa referência. Esta linha indica a raiz do projeto como referência, e as variáveis passam a ser encontradas em qualquer comando.
- **`interpreter_python = auto_silent`:** o Ansible detecta o Python da máquina automaticamente, sem exibir avisos sobre isso.
- **`retry_files_enabled = False`:** desativa a criação de arquivos `.retry` quando uma execução falha. Esses arquivos poluem a pasta do projeto.

Verificando qual configuração está em uso

```
ansible --version
```

A linha `config file = ...` deve apontar para o `ansible.cfg` do projeto, e não para `None`.

O Ansible procura o `ansible.cfg` apenas no diretório atual, sem subir para os diretórios pais. Um comando executado de dentro de `inventory/`, por exemplo, não encontra a configuração do projeto. Por isso, os comandos são executados a partir da raiz do projeto.

Para listar apenas as configurações que diferem do padrão:

```
ansible-config dump --only-changed
```

Cada linha mostra o nome interno de uma opção, o arquivo de onde ela veio e o valor. As quatro opções do arquivo devem aparecer. Os nomes internos nem sempre coincidem com os nomes usados no `ansible.cfg`: a opção `inventory`, por exemplo, aparece como `DEFAULT_HOST_LIST`. Para descobrir a correspondência, consulte `ansible-config list`.

> **Referência:** Ansible. Configuring Ansible. docs.ansible.com. Lista de opções do `ansible.cfg` e ordem de precedência entre fontes de configuração.

3.2 Inventário: quais máquinas existem

O inventário lista as máquinas gerenciadas pelo Ansible, organizadas em **grupos**, e guarda as variáveis de cada máquina. Ele fica em `inventory/hosts.yml`.

Há uma regra de nomes que sustenta toda a configuração: **o nome de cada host no inventário é o nome curto da máquina**, o que `hostname -s` retorna. O Slurm também identifica cada nó pelo nome da máquina, então usar o mesmo nome nos dois lugares elimina uma fonte de erro.

Exemplo com uma única máquina, usada ao mesmo tempo como controlador e como nó de computação:

```
all:
  children:
    controller:
      hosts:
        <hostname>:
          ansible_connection: local
    compute:
      hosts:
        <hostname>:
          ansible_connection: local
          cpus: 8
```

```
        real_memory: 7800
        gpu_count: 1
        gpu_device: /dev/nvidia0
```

Leitura de dentro para fora:

- `all` é o grupo que contém todos os outros. `children` lista os subgrupos.
- `controller` e `compute` são grupos. Um host pode pertencer a mais de um grupo, e é por isso que a mesma máquina aparece nos dois.
- `<hostname>` é um marcador. Substitua pelo resultado de `hostname -s`.
- `ansible_connection: local` executa as tarefas na própria máquina, sem SSH.
- `cpus`, `real_memory`, `gpu_count` e `gpu_device` são **variáveis de host**. Elas pertencem à máquina, então ficam junto dela. Foram definidas no grupo `compute` porque só os nós de computação as usam. Os valores são ilustrativos.

Por que separar em grupos, se a máquina é a mesma? As roles usam os grupos para decidir o que fazer em cada host. O controlador só é iniciado em hosts do grupo `controller`, e o serviço de nó só nos do grupo `compute`. Quando houver máquinas diferentes, a lógica é a mesma: muda apenas a lista de hosts de cada grupo.

Verificando o inventário

Para ver a estrutura, como o Ansible a interpreta:

```
ansible-inventory --graph
```

Saída ilustrativa:

```
@all:
  |--@controller:
  |  |--node01
  |--@compute:
  |  |--node01
```

Para ver as variáveis de um host:

```
ansible-inventory --host <hostname>
```

Os valores de `cpus`, `real_memory`, `gpu_count` e `gpu_device` devem aparecer com os números que você definiu.

> **Referência:** Ansible. How to build your inventory. docs.ansible.com, seções sobre inventários estáticos em YAML e variáveis de host.

3.3 Variáveis: a fonte única da verdade

Variáveis são valores nomeados que as tarefas e os templates usam. Para que a configuração não se repita, cada valor deve existir em um único lugar. Neste tutorial há dois lugares para isso:

- **`group_vars/all.yml`** guarda valores que valem para o cluster inteiro.
- **O inventário** guarda valores que pertencem a cada máquina.

Valores do cluster: `group_vars/all.yml`

O nome `all` indica que as variáveis valem para todos os hosts. O Ansible carrega esse arquivo sozinho, porque a pasta `group_vars/` está na pasta de referência do projeto (seção 3.1).

```
cluster_name: mini-cluster
slurmd_port: 6818
```

O módulo `debug` mostra o valor de uma variável sem alterar nada. O padrão `all` indica que o comando vale para todos os hosts do inventário:

```
ansible all -m ansible.builtin.debug -a "var=cluster_name"
```

Saída ilustrativa:

```
<hostname> | SUCCESS => {
    "cluster_name": "mini-cluster"
}
```

Se a saída mostrar `VARIABLE IS NOT DEFINED!`, o Ansible não encontrou o arquivo. Confira a linha `playbook_dir` do `ansible.cfg` (seção 3.1).

Topologia: duas formas de declarar os nós

Os nós do cluster podem ser declarados de duas formas. Ambas funcionam com uma máquina. A diferença aparece quando há várias, e por isso as duas são apresentadas.

Abordagem A: lista de nós em `group_vars`

Nesta forma, `group_vars/all.yml` contém uma lista com um item por nó:

```
controller_host: <hostname>

slurm_nodes:
  - name: <hostname>
    addr: 127.0.0.1
    cpus: 8
    real_memory: 7800
    gpu_count: 1
    gpu_device: /dev/nvidia0
```

A forma é simples e fácil de ler, porque toda a topologia está em um único arquivo. O limite aparece quando o cluster cresce. Cada máquina precisa ser descrita duas vezes: no inventário, para existir como host, e na lista `slurm_nodes`, para existir como nó. Nada verifica se as duas descrições batem. Se uma máquina for adicionada a uma das listas e esquecida na outra, a configuração gerada fica errada sem nenhum aviso.

Os nomes dos itens devem ser literais, como acima. Não use `"{{ ansible_hostname }}"`: esse valor é coletado de cada host e muda conforme o host processado, o que faz a lista variar entre máquinas.

Abordagem B: lista derivada do inventário (recomendada)

Nesta forma, a topologia não é repetida. `group_vars/all.yml` tem apenas os valores do cluster:

```
cluster_name: mini-cluster
slurmd_port: 6818
controller_host: "{{ groups['controller'][0] }}"
```

A lista de nós vem do próprio inventário, e o hardware de cada nó vem das variáveis do host correspondente. Os templates fazem essa leitura com duas construções:

- **`groups['compute']`** é a lista de hosts do grupo `compute`. Ela é a mesma, qualquer que seja o host que está sendo processado.
- **`hostvars[nome]`** permite ler as variáveis de qualquer host, a partir de qualquer outro. Por exemplo, `hostvars[h].cpus` lê o número de CPUs do host `h`.

O valor de `controller_host` também não depende do host processado. `groups` é global, então `groups['controller'][0]` resolve para o mesmo nome em todas as máquinas. Essa é a diferença essencial em relação ao `ansible_hostname`, que muda de uma máquina para outra.

Quando a topologia cresce, o trabalho se resume a adicionar a máquina ao inventário, com suas variáveis. Nenhum outro arquivo precisa ser editado.

Quadro comparativo

Critério	**Abordagem A: lista em `group_vars`**	**Abordagem B: derivada do inventário**
Onde a topologia fica	Em `group_vars/all.yml`, separada das máquinas	No inventário, junto de cada máquina
Esforço para adicionar uma máquina	Descrever a máquina em dois lugares	Adicionar a máquina ao inventário
Risco de divergência	Alto: as duas descrições podem não bater	Baixo: há uma única fonte
Dependência de fatos coletados	Nenhuma, se os nomes forem literais	Nenhuma

Critério	Abordagem A: lista em `group_vars`	Abordagem B: derivada do inventário
Legibilidade para quem começa	Alta, tudo em um arquivo	Média: exige entender grupos e `hostvars`
Adequação a várias máquinas	Funciona, com disciplina manual	Adequada por construção

A abordagem A é uma boa forma de começar, porque mostra a topologia de maneira direta. A abordagem B é a recomendada, porque remove a repetição que causa a divergência. A recomendação segue a orientação da documentação oficial do Ansible, que trata o inventário como a fonte da topologia e as variáveis de host como o lugar dos valores que pertencem a cada máquina.

> **Referência:** Ansible. Using variables. docs.ansible.com. Seções sobre `group_vars`, `host_vars` e precedência de variáveis. **Referência:** Ansible. Magic variables, and how to access information about other hosts. docs.ansible.com. Explica `groups`, `group_names` e `hostvars`. **Referência:** Ansible. Variables: Facts. docs.ansible.com. Explica as variáveis coletadas automaticamente, como `ansible_hostname`, e por que elas dependem do host.

3.4 Playbook: a receita de configuração

O playbook é o arquivo que diz **quais roles aplicar, em quais hosts e com quais privilégios**. Por convenção, chama-se `site.yml`.

```
- name: Configuração mínima do cluster
  hosts: all
  become: true
  roles:
    - base
    - munge
    - slurm
```

Leitura:

- **`name:`** descrição livre, exibida na saída da execução.
- **`hosts: all`:** aplica o playbook a todos os hosts do inventário.
- **`become: true`:** equivale a usar `sudo`. Necessário para instalar pacotes e editar arquivos em `/etc/`.
- **`roles:`** lista de roles, executadas **na ordem em que aparecem**. A ordem importa: `base` instala os pacotes, `munge` depende deles, e `slurm` depende dos dois.

O que é uma role

Uma role é uma pasta com estrutura fixa. O Ansible procura cada parte no lugar previsto:

```
roles/slurm/
├── tasks/main.yml        # o que fazer (lista de tarefas)
├── handlers/main.yml     # o que fazer quando algo muda
└── templates/            # arquivos com variáveis, gerados no host
    ├── slurm.conf.j2
    ├── gres.conf.j2
    └── cgroup.conf.j2
```

Ao ver `roles: - slurm`, o Ansible lê `roles/slurm/tasks/main.yml`, e assim por diante.

Verificando o playbook

Confira a sintaxe antes de aplicar qualquer coisa:

```
ansible-playbook site.yml --syntax-check
```

Saída esperada: `playbook: site.yml`. Erros de indentação aparecem aqui, com o número da linha.

Para listar as tarefas que seriam executadas, sem executá-las:

```
ansible-playbook site.yml --list-tasks
```

Nesta etapa, as tarefas ainda estão vazias (`[]`). A saída mostra o nome do playbook, mas nenhuma tarefa abaixo de `tasks:`. É o comportamento esperado.

> **Referência:** Ansible. Playbooks. docs.ansible.com. Seções sobre `hosts`, `become` e ordem de execução. **Referência:** Ansible. Using roles. docs.ansible.com. Estrutura de diretórios de roles.

3.5 Tarefas e módulos

Cada tarefa dentro de uma role usa um **módulo**: uma função pronta do Ansible. Você descreve o resultado desejado, e o módulo cuida de como alcançá-lo. Os módulos mais usados nesta apostila:

Módulo	Para que serve	Exemplo
`apt`	Instalar ou remover pacotes	Instalar o `slurm-wlm`
`file`	Criar pastas e ajustar permissões e donos	Criar `/var/spool/slurmd`
`template`	Gerar um arquivo a partir de um `.j2`	Gerar `slurm.conf`
`systemd`	Iniciar, parar ou habilitar serviços	Iniciar `slurmctld`

Exemplo de tarefa, na forma que aparece nas próximas partes:

```
- name: Instalar munge
  ansible.builtin.apt:
```

```
    name: munge
    state: present
```

Cada parte tem um significado:

- **`name:`** texto exibido na saída. Escreva algo que faça sentido quando você ler o log.
- **`ansible.builtin.apt:`** o módulo. O prefixo `ansible.builtin` indica que ele vem junto com o Ansible.
- **`name: munge`:** argumento do módulo, o nome do pacote.
- **`state: present`:** estado desejado. Pode ser `present` (instalado), `absent` (removido) ou `latest` (versão mais recente).

Documentação de um módulo, direto no terminal:

```
ansible-doc ansible.builtin.apt
```

Você verá os parâmetros aceitos e exemplos de uso. É o caminho mais rápido para descobrir o que um módulo faz, sem decorar nada.

Exercício de leitura

Sem executar nada, responda: qual combinação de parâmetros garante que um serviço esteja em execução e seja iniciado automaticamente no boot? Dica: os parâmetros estão em `ansible-doc ansible.builtin.systemd`.

> **Referência:** Ansible. Module Index. docs.ansible.com. Índice de módulos, com a documentação de cada um.

3.6 Idempotência: executar de novo sem efeito colateral

Idempotência é a propriedade mais importante de uma automação de infraestrutura. Uma operação é idempotente quando, aplicada várias vezes, produz o mesmo resultado final que na primeira vez.

Teste com um módulo inofensivo, que cria uma pasta em `/tmp`:

```
ansible localhost -m ansible.builtin.file -a "path=/tmp/apostila-demo stat
```

Execute o mesmo comando de novo:

```
ansible localhost -m ansible.builtin.file -a "path=/tmp/apostila-demo stat
```

Compare a saída:

- Na **primeira** execução: `changed` (a pasta foi criada).
- Na **segunda** execução: `ok` (a pasta já existia, então nada mudou).

O módulo verifica o estado atual e age apenas se for necessário.

Agora um comando que **não** é idempotente:

```
ansible localhost -m ansible.builtin.command -a "mkdir /tmp/apostila-demo-
```

Execute duas vezes. Na segunda, o `mkdir` falha, porque a pasta já existe. O módulo `command` executa o que for pedido, sem verificar o estado. Por isso, a apostila recomenda evitar `command` e `shell` sempre que existir um módulo específico.

Limpeza:

```
rm -rf /tmp/apostila-demo /tmp/apostila-demo-2
```

Simulação antes de aplicar

O modo `--check` simula a execução sem alterar nada e mostra o que mudaria:

```
ansible-playbook site.yml --check -K
```

O playbook declara `become: true` (seção 3.4), então até a simulação é executada com privilégio de administrador. A opção `-K` (forma curta de `--ask-become-pass`) faz o Ansible pedir a senha do `sudo` antes de começar. Sem ela, a execução falha logo na coleta de fatos, com a mensagem `sudo: a password is required`.

Com as roles ainda vazias, a simulação executa apenas a coleta de fatos. Nas próximas partes, ela será a primeira verificação antes de qualquer execução real.

> **Referência:** Ansible. Validating tasks: check mode and diff mode. docs.ansible.com. **Referência:** Ansible. Understanding privilege escalation: become. docs.ansible.com. Explica `become` e a opção `--ask-become-pass`. **Referência:** Morris, K. Infrastructure as Code: Managing Servers in the Cloud. O'Reilly. Capítulos sobre idempotência e a diferença entre scripts imperativos e declarações de estado. (Verifique a edição antes de citar.)

Próxima: Parte 4, pacotes e autenticação.

Parte 4: Pacotes e autenticação

Objetivo desta parte: instalar os pacotes do Slurm e do munge pelo playbook, criar os diretórios de estado e garantir que a autenticação entre os componentes funcione.

Pré-requisito: Parte 3 concluída. O playbook `site.yml` passa na checagem de sintaxe e a lista de tarefas aparece vazia.

Esta é a primeira parte que altera o sistema. Antes de cada execução real, rode a simulação com `--check`, como indicado na seção 3.6. Os comandos desta parte são executados a partir da raiz do projeto `slurm-lab`.

4.1 Instalação de pacotes (role base)

A role `base` é a primeira da lista do playbook. Ela instala os pacotes que todas as máquinas do cluster precisam e prepara os diretórios em que o Slurm guarda seu estado.

Os três pacotes têm papéis distintos:

Pacote	Papel
`slurm-wlm`	Contém os daemons `slurmctld` (controlador) e `slurmd` (nó de computação), além de `slurmdbd` (histórico, usado mais adiante).
`slurm-client`	Contém os comandos que o usuário usa, como `sinfo`, `squeue`, `sbatch` e `srun`.
`munge`	Contém o serviço de autenticação e gera a chave compartilhada entre as máquinas.

Os nomes dos pacotes podem mudar entre versões da distribuição. Para confirmar que existem na sua máquina, use `apt policy <nome>`, como visto na Parte 2.

O arquivo `roles/base/tasks/main.yml` fica assim:

```
- name: Instalar pacotes do Slurm e do munge
  ansible.builtin.apt:
    name:
      - slurm-wlm
      - slurm-client
      - munge
    state: present
    update_cache: true

- name: Criar diretório de estado do controlador
  ansible.builtin.file:
    path: /var/spool/slurmctld
    state: directory
```

```
    owner: slurm
    group: slurm
    mode: "0755"

- name: Criar diretório de spool do nó de computação
  ansible.builtin.file:
    path: /var/spool/slurmd
    state: directory
    owner: root
    group: root
    mode: "0755"
```

Leitura da tarefa de instalação:

- **`update_cache: true`** atualiza a lista de pacotes antes de instalar, equivalente a `sudo apt update`. Sem isso, o Ansible pode tentar instalar uma versão antiga da lista local.
- **`state: present`** garante que os pacotes estejam instalados. Se já estiverem, nada muda. Esse comportamento é o que torna a tarefa idempotente (seção 3.6).
- A lista de `name` aceita vários pacotes em uma única chamada, o que é mais rápido e mais claro do que uma tarefa por pacote.

  Atenção: a instalação altera pacotes do sistema. Antes de executar, rode a simulação:

  ```
  bash ansible-playbook site.yml --check -K
  ```

  Na simulação, o Ansible mostra as tarefas que mudariam sem aplicá-las. Se algo não fizer sentido, interrompa antes da execução real.

Execução real, com a senha do `sudo` solicitada no início:

```
ansible-playbook site.yml -K
```

Nesta etapa, o playbook executa a role `base` e as roles `munge` e `slurm`. As duas últimas ainda estão vazias, então não fazem nada. Isso é esperado.

Por que os diretórios têm donos diferentes

Os diretórios de estado do Slurm têm finalidades diferentes, e por isso não recebem o mesmo dono:

- **`/var/spool/slurmctld`** guarda o estado do controlador: a fila, os jobs em andamento e as alocações. Esse diretório pertence ao usuário `slurm`, que é o usuário sob o qual o `slurmctld` roda. Se o dono estiver errado, o controlador não consegue gravar o próprio estado e não inicia.
- **`/var/spool/slurmd`** guarda arquivos temporários do nó de computação. O `slurmd` roda como `root`, porque precisa criar isolamentos de processo e dispositivos. Por isso, o dono desse diretório é `root`.

A diferença entre `owner`, `group` e `mode` é a base das permissões de arquivos no Linux. Você pode ler as permissões de um diretório assim:

```
ls -ld /var/spool/slurmctld /var/spool/slurmd
```

A primeira coluna mostra as permissões (por exemplo, `drwxr-xr-x`), e as colunas seguintes mostram o dono e o grupo.

> **Referência:** Ubuntu. Ubuntu Packages Search. packages.ubuntu.com. Permite verificar nomes de pacotes e versões por distribuição. **Referência:** SchedMD. Slurm Quick Start Administrator Guide. slurm.schedmd.com/quickstart_admin.html. Descreve os daemons e os diretórios de estado.

4.2 Diretórios de estado e permissões

A tarefa de criação de diretório usa o módulo `file`. Esse módulo descreve o estado desejado de um caminho: se ele existe, qual o tipo, quem é o dono e quais são as permissões. Ele verifica o estado atual antes de agir, o que o torna idempotente.

Os campos da tarefa têm estes significados:

- **`path`:** o caminho do diretório.
- **`state: directory`:** o caminho deve ser um diretório. Se não existir, é criado.
- **`owner` e `group`:** quem é o dono e o grupo do diretório.
- **`mode: "0755"`:** permissões em notação octal. O primeiro dígito é o dono, o segundo o grupo e o terceiro os demais. Cada dígito soma os valores 4 (leitura), 2 (escrita) e 1 (execução). Assim, 7 significa leitura, escrita e execução, e 5 significa leitura e execução.

Por que as aspas em `"0755"`? Sem elas, o YAML pode interpretar o número como decimal, e a permissão sai errada. Sempre use aspas em permissões.

Se você alterar o dono de um diretório manualmente e executar o playbook de novo, o Ansible corrige o dono de volta para o valor declarado. Essa é a forma de manter a configuração consistente com o que está no repositório.

> **Referência:** Ansible. ansible.builtin.file module. docs.ansible.com. Parâmetros `path`, `state`, `owner`, `group` e `mode`. **Referência:** Kerrisk, M. The Linux Programming Interface. No Starch Press. Capítulo sobre permissões de arquivos e diretórios. (Verifique a edição antes de citar.)

4.3 munge: chave e serviço (role `munge`)

O munge autentica as mensagens trocadas entre os componentes do Slurm. Ele depende de uma **chave** compartilhada, guardada em `/etc/munge/munge.key`. Três fatos sobre essa chave orientam a configuração:

1. O pacote `munge` gera a chave automaticamente na instalação.
2. Todas as máquinas do cluster precisam ter **a mesma chave**. Com uma única máquina, isso é automático. Quando houver várias, a distribuição da chave é tratada na Parte 10.
3. A chave é um segredo. Quem a possui pode se passar por qualquer componente do cluster. Por isso, ela tem permissão restrita e **nunca deve ser copiada para o repositório do projeto**.

   Atenção: o arquivo `munge.key` não deve entrar no controle de versão. Se ele aparecer em `git status`, remova-o do índice antes de qualquer commit e revise o que já foi enviado a repositórios remotos.

O arquivo `roles/munge/tasks/main.yml` fica assim:

```
- name: Garantir que o serviço munge está ativo
  ansible.builtin.systemd:
    name: munge
    state: started
    enabled: true

- name: Garantir dono e permissões da chave do munge
  ansible.builtin.file:
    path: /etc/munge/munge.key
    owner: munge
    group: munge
    mode: "0400"
```

Leitura:

- **`systemd` com `state: started` e `enabled: true`** faz o serviço rodar agora e ser iniciado no boot. Os dois valores são independentes: `started` trata do agora, e `enabled` trata do próximo boot.
- **`file` com `mode: "0400"`** restringe a chave a leitura apenas pelo dono. Nenhum outro usuário pode ler o arquivo.
- A tarefa de permissões não cria a chave. Ela apenas garante que, se a chave existir, o dono e as permissões estejam corretos. Se a chave não existir, a tarefa falha, o que é o comportamento desejado: sem chave, o munge não funciona e o problema precisa aparecer cedo.

Execute a simulação e depois a aplicação:

```
ansible-playbook site.yml --check -K
ansible-playbook site.yml -K
```

Confira o resultado com `ls`:

```
sudo ls -l /etc/munge/munge.key
```

A saída deve mostrar `-r--------` como permissão e `munge munge` como dono e grupo.

> **Referência:** Dun, C. MUNGE Uid 'N' Gid Emporium. Documentação oficial do projeto. dun.github.io/munge.
> **Referência:** Ansible. ansible.builtin.systemd module. docs.ansible.com. Parâmetros `state` e `enabled`.

4.4 Testando a autenticação com `munge -n | unmunge`

Até aqui, a configuração foi aplicada. Agora é preciso verificar se ela funciona. Esta seção faz dois testes: a autenticação do munge e a leitura do hardware pelo Slurm.

Teste de autenticação

O comando abaixo cria uma credencial e a valida em seguida, sem envolver o Slurm:

```
munge -n | unmunge
```

Saída esperada (resumida):

```
STATUS:          Success (0)
ENCODE_HOST:     nome-da-maquina
UID:             usuario
```

`munge -n` cria a credencial, e `unmunge` a valida. Se aparecer `Success`, a chave local e o serviço estão corretos. Se aparecer um erro de conexão, o serviço não está ativo. Verifique com:

```
systemctl status munge
```

Se o serviço estiver ativo e a validação falhar, o problema mais comum é uma chave diferente entre máquinas. Com uma única máquina, isso não ocorre, mas o diagnóstico vale para a Parte 10.

Verificando o hardware visto pelo Slurm

O pacote `slurm-wlm` foi instalado na seção 4.1. Ele inclui o comando `slurmd -C`, que mostra a configuração de hardware que o Slurm enxerga:

```
slurmd -C
```

Saída ilustrativa (resumida):

```
NodeName=nome-da-maquina CPUs=8 Boards=1 SocketsPerBoard=1 CoresPerSocket=
UpTime=0-01:23:45
```

Compare com os valores que você levantou na Parte 2:

- **CPUs:** o valor de `CPUs=` deve corresponder ao de `nproc`.
- **Memória:** o valor de `RealMemory=` é a memória que o Slurm vai usar. Ele costuma ser um pouco menor que o total de `free -m`, porque parte da memória é reservada pelo kernel.

Se houver diferença, prevalece o valor de `slurmd -C`. Atualize `cpus` e `real_memory` no inventário, no host correspondente, com o `RealMemory` como limite. Declarar mais memória do que a máquina tem é o erro mais comum, e ele faz o nó aparecer como indisponível na Parte 9.

> **Atenção:** os serviços `slurmctld` e `slurmd` podem já estar instalados e tentar iniciar com a configuração padrão do pacote. Nesta etapa, o projeto ainda não tem configuração própria do Slurm. Se `systemctl status slurmctld` mostrar falha, isso é esperado. A configuração correta é aplicada na Parte 8.
>
> **Referência:** SchedMD. slurmd: Slurm compute node daemon. Página de manual, disponível em slurm.schedmd.com. Descreve as opções, incluindo `-C`.

Resumo da Parte 4

Ao final, você terá: os pacotes do Slurm e do munge instalados pelo playbook, os diretórios de estado com donos e permissões corretos, a chave do munge protegida, e a confirmação de que o munge autentica e de que o Slurm enxerga o hardware corretamente.

Nenhum serviço do Slurm foi configurado ainda. Essa configuração começa na Parte 5, com o estudo de cada componente, e é aplicada pelo playbook na Parte 8.

Próxima: Parte 5, Slurm, componente por componente.

Parte 5: Slurm, componente por componente

Objetivo desta parte: entender cada componente do Slurm, o papel de cada arquivo de configuração e as decisões de escalonamento que o arquivo principal controla, antes de gerar esse arquivo com o Ansible.

Pré-requisito: Parte 4 concluída. Os pacotes do Slurm e do munge estão instalados, e o munge autentica localmente.

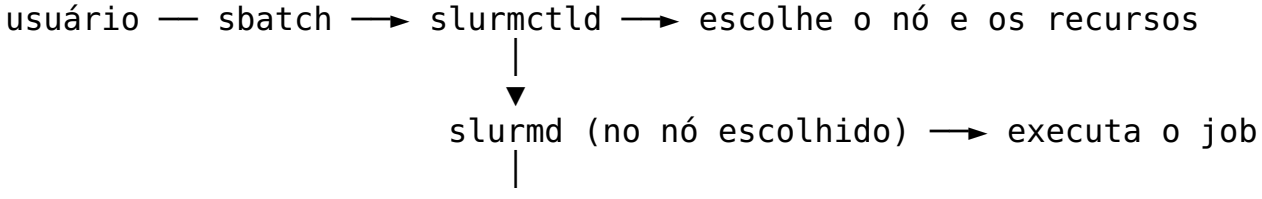
Esta parte é de estudo. Os exemplos de configuração são ilustrativos e não são aplicados ao sistema. A geração do arquivo real acontece na Parte 7, por meio de templates. Nenhum arquivo do Slurm deve ser editado à mão.

5.1 Arquitetura: `slurmctld`, `slurmd`, `slurmdbd` e os comandos do usuário

O Slurm é composto por processos (chamados daemons) que rodam em segundo plano, e por comandos que o usuário digita. Cada daemon tem um papel e fica em um tipo de máquina diferente.

Componente	Tipo	Onde roda	Papel
`slurmctld`	Daemon	Controlador	Mantém a fila de jobs, decide quais recursos cada job recebe e acompanha o estado de todos os nós.
`slurmd`	Daemon	Cada nó de computação	Recebe ordens do controlador, executa os jobs e informa ao controlador o estado do próprio nó.
`slurmdbd`	Daemon	Controlador ou máquina dedicada	Grava o histórico de jobs em um banco de dados. É opcional e será usado na Parte 10.
`sinfo`, `squeue`, `sbatch`, `srun`, `scancel`, `sacct`	Comandos	Qualquer máquina do cluster	Consultam o estado, enviam jobs e cancelam jobs.

Um fluxo típico, de quando o usuário envia um job até o término:





O usuário nunca fala diretamente com o `slurmd`. Todo pedido passa pelo controlador, que é o único componente com visão completa do cluster. Essa centralização é o que permite à fila funcionar como uma fila única.

Como verificar se um daemon está ativo

Os daemons são serviços do systemd, e o diagnóstico começa pelo mesmo caminho de qualquer outro serviço:

```
systemctl status slurmctld
systemctl status slurmd
```

Para ler os registros de um serviço, use o `journalctl`, detalhado na Parte 11:

```
journalctl -u slurmctld --no-pager -n 20
```

Quando um daemon falha ao iniciar, é comum que o motivo esteja no registro, e não na linha de status. O comando acima mostra as últimas vinte linhas do registro.

Modo de primeiro plano

Para depurar um daemon, é útil rodá-lo no terminal, em vez de em segundo plano. A opção `-D` mantém o processo em primeiro plano, e `-vvv` aumenta o nível de detalhe das mensagens. Exemplo com o `slurmd`:

```
sudo slurmd -D -vvv
```

Para interromper, use `Ctrl+C`. Esse modo é usado apenas para diagnóstico, porque o serviço do systemd precisa estar parado para não haver dois processos disputando a mesma porta. Pare o serviço antes:

```
sudo systemctl stop slurmd
```

> **Referência:** SchedMD. Slurm Quick Start Administrator Guide. slurm.schedmd.com/quickstart_admin.html. Descreve a arquitetura de daemons e as máquinas em que cada um roda.
> **Referência:** Yoo, A. B.; Jette, M. A.; Grondona, M. SLURM: Simple Linux Utility for Resource Management. In: Job Scheduling Strategies for Parallel Processing (JSSPP), Springer, 2003. Artigo científico que descreve a arquitetura original do Slurm.

5.2 `slurm.conf`: o arquivo principal de configuração

O arquivo `/etc/slurm/slurm.conf` é lido por todos os daemons e comandos do Slurm. Ele descreve o cluster inteiro: quem são as máquinas, quais recursos cada uma tem, como as filas são organizadas e como o escalonador decide. Como todos os componentes leem o mesmo arquivo, ele precisa ser idêntico em todas as máquinas do cluster.

As opções do arquivo se agrupam em áreas. A tabela abaixo mostra as áreas que esta parte explica, com o nome da opção e o assunto da seção em que ela é detalhada:

Área	Exemplos de opções	Detalhado em
Identificação	`ClusterName`, `SlurmctldHost`	Esta seção
Usuário e caminhos	`SlurmUser`, `StateSaveLocation`, `SlurmdSpoolDir`	Esta seção
Processos e isolamento	`ProctrackType`, `TaskPlugin`	Seção 5.4 e Parte 6
Escalonador	`SchedulerType`, `SelectType`, `SelectTypeParameters`	Seção 5.4
Recursos genéricos	`GresTypes`	Parte 6
Histórico	`AccountingStorageType`, `JobAcctGatherType`	Parte 10
Nós e partições	`NodeName`, `PartitionName`	Seção 5.3

Um trecho ilustrativo do início do arquivo, para entender o formato:

```
ClusterName=mini-cluster
SlurmctldHost=nome-da-maquina
SlurmUser=slurm
StateSaveLocation=/var/spool/slurmctld
SlurmdSpoolDir=/var/spool/slurmd
```

Leitura:

- **`ClusterName`** é um nome livre para o cluster. Aparece em relatórios e em algumas mensagens de log.
- **`SlurmctldHost`** informa em qual máquina o controlador roda. O valor deve ser o nome real da máquina, o mesmo retornado por `hostname -s`. Se o nome estiver diferente, o `slurmctld` não reconhece a própria máquina como controlador e não inicia.
- **`SlurmUser`** é o usuário sob o qual o `slurmctld` roda. É por isso que o diretório de estado, na Parte 4, pertence a esse usuário.
- **`StateSaveLocation`** é o diretório onde o controlador grava a fila e o estado dos jobs. Se esse diretório for apagado, a fila em andamento é perdida.
- **`SlurmdSpoolDir`** é o diretório de trabalho temporário do nó de computação.

Formato geral do arquivo: cada linha define uma opção no formato `Nome=valor`. Opções são case-sensitive, ou seja, `ClusterName` e `clustername` são diferentes. Comentários começam com #.

Para consultar o significado de qualquer opção, use o manual do arquivo:

```
man slurm.conf
```

> **Atenção:** o arquivo `slurm.conf` será gerado pelo Ansible a partir de um template (Parte 7). Edições feitas diretamente no arquivo gerado são sobrescritas na próxima execução do playbook. Para alterar a configuração, altere a fonte, que são as variáveis do inventário e de `group_vars/all.yml`.
>
> **Referência:** SchedMD. slurm.conf: Slurm configuration file. slurm.schedmd.com/slurm.conf.html. Documentação completa de todas as opções.

5.3 Nós (`NodeName`) e partições (`PartitionName`)

Dois conceitos organizam o cluster dentro do `slurm.conf`:

- **Nó** é uma máquina de computação, descrita pela linha `NodeName`. A linha informa quanto hardware a máquina tem.
- **Partição** é um grupo de nós com regras comuns, descrito pela linha `PartitionName`. Um usuário escolhe em qual partição o job será enviado, e a partição limita quais nós podem recebê-lo.

Um exemplo ilustrativo com um nó e uma partição:

```
NodeName=node01 NodeAddr=127.0.0.1 Port=6818 CPUs=8 RealMemory=7900 Gres=g
PartitionName=debug Nodes=node01 Default=YES MaxTime=INFINITE State=UP
```

Leitura da linha de nó:

- **`NodeName`** é o nome do nó. Deve coincidir com o nome que o próprio nó usa para se identificar.
- **`NodeAddr`** e **`Port`** indicam onde o controlador encontra o `slurmd` desse nó. Em uma máquina só, o endereço é `127.0.0.1`, que aponta para a própria máquina.
- **`CPUs`** e **`RealMemory`** são os recursos declarados. Como visto na Parte 4, eles não devem exceder o que a máquina realmente tem.
- **`Gres=gpu:1`** declara uma GPU. A declaração do dispositivo físico fica em outro arquivo, detalhado na Parte 6.
- **`State=UNKNOWN`** é o estado inicial. O controlador troca esse estado para `IDLE` quando o `slurmd` do nó responde corretamente.

Leitura da linha de partição:

- **`Nodes=node01`** lista os nós que pertencem à partição. Pode ser uma lista separada por vírgulas.

- **`Default=YES`** faz desta a partição usada quando o usuário não escolhe outra.
- **`MaxTime=INFINITE`** não limita o tempo de execução dos jobs. Em clusters compartilhados, costuma-se usar um limite, como `MaxTime=24:00:00`. Esse ajuste é uma decisão de política do cluster, e não um valor técnico obrigatório.
- **`State=UP`** deixa a partição aceitando jobs. Com `DOWN`, a partição recusa novos envios.

Dois nomes diferentes para dois conceitos diferentes, e uma regra prática: o nó é o hardware, e a partição é a regra de acesso ao hardware. Um mesmo nó pode pertencer a mais de uma partição.

> **Referência:** SchedMD. slurm.conf: NodeName and PartitionName. slurm.schedmd.com/slurm.conf.html. Seções sobre nós e partições.

5.4 Escalonador e seleção de recursos

Esta seção trata de três decisões que o `slurm.conf` controla: quando um job começa, quais recursos ele recebe e como o Slurm rastreia os processos de cada job.

Escalonador: `SchedulerType`

Um escalonador decide a ordem em que os jobs da fila começam. Duas políticas básicas são importantes:

- **FIFO (first in, first out):** o primeiro job da fila é o primeiro a começar. É simples e justo na ordem de chegada, mas um job grande pode bloquear a fila inteira enquanto espera recursos que demorarão a liberar.
- **Backfill:** permite que jobs menores, que cabem no tempo livre antes de um job grande começar, sejam executados antes dele, sem atrasá-lo. Isso aumenta o uso dos recursos.

A linha correspondente no arquivo é:

```
SchedulerType=sched/backfill
```

O valor `sched/backfill` ativa a política de backfill. A barra separa o tipo de plugin do nome do plugin, um padrão que se repete em várias opções do Slurm.

Seleção de recursos: `SelectType` e `SelectTypeParameters`

A seleção de recursos decide quais partes de cada nó um job recebe. Duas opções definem isso:

```
SelectType=select/cons_tres
SelectTypeParameters=CR_Core
```

- **`select/cons_tres`** trata CPUs, memória e recursos genéricos (como GPUs) como recursos consumíveis. Um job reserva o que pediu, e o restante continua disponível para outros jobs. É a opção que permite declarar GPUs como recurso (Parte 6).
- **`CR_Core`** significa que a unidade de alocação de CPU é o núcleo físico. Um job que pede uma CPU recebe um núcleo inteiro, e não uma parte dele. Essa escolha evita que dois jobs disputem o mesmo núcleo.

Rastreamento de processos: `ProctrackType` e `TaskPlugin`

Para cancelar um job e garantir que nenhum processo dele sobreviva, o Slurm precisa identificar todos os processos que pertencem a esse job. Duas opções tratam disso:

```
ProctrackType=proctrack/cgroup
TaskPlugin=task/cgroup,task/affinity
```

- **`proctrack/cgroup`** identifica os processos de um job por meio de cgroups (apresentados na Parte 6). Um processo que tente escapar do grupo continua sendo rastreado.
- **`task/cgroup`** aplica os limites de recursos do cgroup a cada tarefa do job.
- **`task/affinity`** vincula cada tarefa a núcleos específicos, o que evita que o sistema operacional mova processos entre núcleos durante a execução.

Retomada após falha: `ReturnToService`

```
ReturnToService=2
```

Essa opção define como um nó que estava indisponível volta a receber jobs. O valor `2` faz o nó voltar quando o `slurmd` se reapresenta ao controlador com a configuração declarada. Os valores `0` e `1` exigem intervenção manual ou aceitam o retorno de nós com configuração diferente da declarada, o que pode esconder erros de hardware. Para este tutorial, `2` é o valor adequado.

> **Referência:** SchedMD. slurm.conf: SchedulerType, SelectType, ProctrackType, TaskPlugin, ReturnToService. slurm.schedmd.com/slurm.conf.html. Descrição de cada opção e dos valores aceitos.
> **Referência:** Feitelson, D. G.; Rudolph, L.; Schwiegelshohn, U.; Sevcik, K. C.; Wong, P. Theory and Practice in Parallel Job Scheduling. In: Job Scheduling Strategies for Parallel Processing (JSSPP), Springer, 1997. Artigo científico sobre a teoria por trás de políticas como FIFO e backfill. (Verifique a referência exata antes de citar.)

Resumo da Parte 5

Ao final, você entende os três daemons do Slurm e os comandos que o usuário usa, sabe o que cada área do `slurm.conf` controla, e conhece a diferença entre nó e partição. Também entende as decisões de escalonamento que o arquivo define: política de fila, seleção de recursos consumíveis e rastreamento de processos.

Nada foi aplicado ao sistema nesta parte. Os valores de exemplo serão substituídos pelos seus valores reais na geração do arquivo, na Parte 7.

Próxima: Parte 6, GPU como recurso agendável.

Parte 6: GPU como recurso agendável

Objetivo desta parte: fazer com que o Slurm reconheça GPUs como recurso que pode ser pedido e reservado, e entender como o isolamento por cgroups garante que cada job use apenas as GPUs que recebeu.

Pré-requisito: Parte 5 concluída. Você conhece as opções `GresTypes`, `Gres`, `SelectType` e `ProctrackType`.

Esta parte é de estudo, e a seção 6.3 descreve um teste de conferência. Os arquivos de exemplo são ilustrativos. A geração dos arquivos reais acontece na Parte 7. Se o sistema não tiver GPU, leia esta parte mesmo assim, porque a lógica se aplica a qualquer recurso genérico, e substitua os exemplos de GPU por outro recurso, como um dispositivo de vídeo ou uma interface de rede.

6.1 GRES: declarando GPUs

Por padrão, o Slurm gerencia CPUs e memória, porque o sistema operacional já informa esses valores. Uma GPU não é reconhecida dessa forma. O Slurm precisa que alguém declare que ela existe, qual é o nome do recurso e onde está o dispositivo.

Essa declaração é feita com um **GRES** (Generic RESource), um recurso genérico com nome livre. Neste tutorial, o nome escolhido é `gpu`. O nome é uma escolha de quem configura: poderia ser `acelerador` ou qualquer outro. O que importa é que o mesmo nome apareça em todos os lugares em que o recurso é mencionado.

A declaração envolve três pontos:

1. **Tipo de recurso no cluster:** `GresTypes` em `slurm.conf` registra o nome do recurso para o cluster inteiro.
2. **Quantidade por nó:** `Gres` em `slurm.conf` informa quantas unidades cada nó possui.
3. **Dispositivo físico por nó:** `gres.conf` aponta o arquivo que representa cada GPU na máquina.

Em `slurm.conf`

```
GresTypes=gpu
NodeName=node01 ... Gres=gpu:1
```

- **`GresTypes=gpu`** registra o tipo. Essa linha precisa vir antes das linhas de nó. Sem ela, o Slurm não reconhece o nome `gpu`, e a linha de nó é rejeitada.

- **`Gres=gpu:1`** declara uma unidade de `gpu` no nó. O formato é `nome:quantidade`. Para duas GPUs, use `Gres=gpu:2`.

Em `gres.conf`

```
NodeName=node01 Name=gpu File=/dev/nvidia0
```

- **`Name=gpu`** deve ser o mesmo nome registrado em `GresTypes`.
- **`File=/dev/nvidia0`** aponta o arquivo de dispositivo da GPU. O caminho depende do fabricante e do sistema. Em placas NVIDIA, o padrão é `/dev/nvidia0`, `/dev/nvidia1` e assim por diante. Em placas AMD, os dispositivos de renderização ficam em `/dev/dri/`. Para descobrir os arquivos disponíveis, use `ls /dev/dri` ou `ls /dev | grep nvidia`.

Se o nó tiver mais de uma GPU, cada uma ganha uma linha, ou um intervalo de nomes pode ser usado. Exemplo com duas placas NVIDIA:

```
NodeName=node01 Name=gpu File=/dev/nvidia[0-1]
```

Quando o dispositivo não pode ser apontado

Em alguns casos, o dispositivo não tem um arquivo próprio que possa ser apontado, como em nós simulados (Parte 10). Nesse caso, a declaração usa `Count` em vez de `File`:

```
NodeName=node02 Name=gpu Count=2
```

Com `Count`, o Slurm controla a quantidade de unidades reservadas, mas não isola nenhum dispositivo real. Essa diferença tem consequências no isolamento, tratadas na seção 6.3.

Detecção automática

O Slurm pode detectar GPUs sozinho, sem listar os arquivos de dispositivo. Para isso, `gres.conf` aceita a opção `AutoDetect`, com valores como `nvml` (NVIDIA) ou `rsmi` (AMD). Essa detecção depende de o pacote do Slurm ter sido compilado com suporte à biblioteca do fabricante. Para saber se a sua instalação tem esse suporte, consulte a documentação da versão instalada antes de usar a opção. Neste tutorial, a declaração explícita com `File` é usada, porque ela é mais fácil de verificar.

> **Referência:** SchedMD. Generic Resource (GRES) Scheduling. slurm.schedmd.com/gres.html. Descreve `GresTypes`, `Gres`, `gres.conf`, `File`, `Count` e `AutoDetect`. **Referência:** SchedMD. gres.conf: Generic Resource configuration file. slurm.schedmd.com/gres.conf.html. Referência das opções do arquivo.

6.2 Pedindo GPUs em um job

Com o recurso declarado, o usuário pede GPUs no momento de enviar o job. Dois formatos são comuns:

Opção	Significado
`--gres=gpu:1`	Pede uma unidade do recurso `gpu` no nó.
`--gpus=1`	Pede uma GPU, em uma sintaxe mais recente que mapeia para o mesmo recurso.

Os dois são válidos, e o resultado é o mesmo no caso simples. A opção `--gres` é mais geral, porque aceita qualquer recurso declarado. A opção `--gpus` é mais específica e mais legível para quem só usa GPUs.

Exemplo de script de job, com comentários explicativos:

```
#!/bin/bash
#SBATCH --job-name=teste-gpu
#SBATCH --partition=debug
#SBATCH --gres=gpu:1
#SBATCH --output=saida_%j.log

hostname
nvidia-smi -L
```

Leitura das diretivas:

- **`--job-name`** dá um nome ao job, exibido em `squeue`.
- **`--partition`** escolhe a partição (Parte 5, seção 5.3).
- **`--gres=gpu:1`** pede uma GPU. Sem essa linha, o job roda sem GPU, mesmo que o nó tenha uma.
- **`--output`** grava a saída em um arquivo. `%j` é substituído pelo número do job.

Para enviar, acompanhar e ver o resultado:

```
sbatch teste-gpu.sh
squeue
cat saida_<numero-do-job>.log
```

Para executar um comando diretamente, sem script, use `srun`. Ele bloqueia o terminal até o comando terminar:

```
srun --gres=gpu:1 nvidia-smi -L
```

Use `srun` para testes rápidos e `sbatch` para trabalhos que podem demorar. A diferença completa entre os dois aparece na Parte 9.

> **Referência:** SchedMD. sbatch: Submit a batch script to Slurm. slurm.schedmd.com/sbatch.html. Descreve as diretivas `#SBATCH` e as opções de recursos. **Referência:** SchedMD. srun: Run parallel

jobs. slurm.schedmd.com/srun.html. Descreve a execução interativa e bloqueante.

6.3 cgroups e isolamento de dispositivos

Declarar uma GPU não impede que um job acesse GPUs que não pediu. Se os processos de um job puderem abrir qualquer arquivo de dispositivo, eles enxergarão todas as placas. O isolamento é tarefa dos **cgroups** (control groups), um recurso do kernel Linux que limita o que cada grupo de processos pode usar.

A configuração do Slurm para cgroups fica em `cgroup.conf`:

```
CgroupPlugin=autodetect
ConstrainCores=yes
ConstrainRAMSpace=yes
ConstrainDevices=yes
```

Cada opção controla um tipo de limite:

- **`CgroupPlugin=autodetect`** deixa o Slurm escolher a versão do cgroup que o kernel oferece. Sistemas recentes usam a versão 2 (cgroup v2), que é a versão unificada.
- **`ConstrainCores=yes`** limita os processos de um job aos núcleos de CPU que ele recebeu.
- **`ConstrainRAMSpace=yes`** limita a memória do job ao valor pedido. Um job que ultrapassar esse valor é encerrado pelo sistema.
- **`ConstrainDevices=yes`** limita os dispositivos de arquivo (como `/dev/nvidia0`) aos que o job recebeu. Esta é a opção que isola as GPUs.

Verificando a versão do cgroup

Antes de depender do isolamento, confirme qual versão de cgroup o sistema usa:

```
stat -fc %T /sys/fs/cgroup
```

A saída `cgroup2fs` indica cgroup v2, a versão unificada. A saída `tmpfs` indica a versão antiga, que tem outro conjunto de recursos. O comando apenas lê o sistema de arquivos e não altera nada.

Para entender o que cada versão oferece, consulte a documentação do kernel (ver referências ao fim da seção).

Como o isolamento de GPU funciona

O isolamento de dispositivos depende de a declaração ter sido feita com `File` (seção 6.1). Com `File`, o Slurm sabe exatamente qual arquivo de dispositivo corresponde a qual GPU, e o cgroup pode liberar apenas esse arquivo para o job. Com `Count`, o Slurm sabe quantas unidades foram

reservadas, mas não tem o arquivo de dispositivo para liberar. Nesse caso, o isolamento de dispositivo não se aplica, e a reserva é apenas contábil.

Esse ponto é importante para a interpretação de resultados: um nó com `Count` pode parecer isolado no `sinfo`, mas o isolamento real só existe com `File`.

Mudanças na configuração de cgroups

Alterações em `cgroup.conf` só têm efeito depois que os serviços do Slurm são reiniciados, e os jobs em andamento podem ser afetados. Por isso, a mudança deve ser feita com a fila vazia, ou com os jobs em andamento já encerrados. A reinicialização é tratada pelo playbook na Parte 8.

> **Atenção:** o arquivo `cgroup.conf` também é gerado pelo Ansible (Parte 7). Não o edite à mão.
>
> **Referência:** SchedMD. cgroup.conf: Slurm cgroup support configuration file. slurm.schedmd.com/cgroup.conf.html. Descreve as opções `ConstrainCores`, `ConstrainRAMSpace` e `ConstrainDevices`. **Referência:** Linux kernel. Control Group v2. docs.kernel.org/admin-guide/cgroup-v2.html. Documentação oficial da versão unificada dos cgroups. **Referência:** Heo, T. Control Group v2. Documentação do kernel Linux, na mesma página acima, com a motivação da versão 2. (Confirme a autoria no documento antes de citar.)

Resumo da Parte 6

Ao final, você entende como uma GPU vira um recurso do Slurm, por meio de três declarações (`GresTypes`, `Gres` e `gres.conf`), e como o usuário a pede no job com `--gres` ou `--gpus`. Também entende que o isolamento real depende de cgroups e da declaração com `File`, e sabe como verificar a versão de cgroup do sistema.

Nada foi aplicado ao sistema nesta parte. Os valores reais serão gerados a partir das variáveis do projeto, na Parte 7, e o teste de isolamento completo será feito na Parte 9.

Próxima: Parte 7, Templates: gerando configuração a partir de variáveis.

Parte 7: Templates: gerando configuração a partir de variáveis

Objetivo desta parte: entender como o Jinja2 transforma as variáveis do projeto em arquivos de configuração do Slurm, e escrever os três templates que a role `slurm` usa.

Pré-requisito: Partes 3 a 6 concluídas. O inventário (`inventory/hosts.yml`) e `group_vars/all.yml` existem e contêm os valores reais da máquina, e você conhece as opções de `slurm.conf`, `gres.conf` e `cgroup.conf`.

Esta parte cria arquivos, mas não os instala. Os templates ficam dentro da role, em `roles/slurm/templates/`. A geração no sistema acontece na Parte 8, quando o playbook os executa. Você pode testar a renderização antes, como mostrado na seção 7.2.

7.1 Jinja2: variáveis, laços e condicionais

Um **template** é um texto que contém espaços reservados. Quando o template é processado, cada espaço reservado é substituído por um valor. O Jinja2 é a linguagem de templates usada pelo Ansible. Ele tem três construções básicas.

Variáveis

A sintaxe `{{ nome }}` insere o valor de uma variável:

```
ClusterName={{ cluster_name }}
```

Se `cluster_name` valer `mini-cluster`, o resultado é `ClusterName=mini-cluster`. Se a variável não existir, o Ansible falha com a mensagem `'cluster_name' is undefined`. Essa falha é desejada: um arquivo com valor vazio ou ausente seria mais difícil de diagnosticar.

Para acessar um campo de um item de dicionário, use ponto:

```
NodeAddr={{ node.addr }}
```

Laços

A construção `{% for %}` repete um trecho para cada item de uma lista:

```
{% for node in nodes %}
NodeName={{ node }} CPUs={{ hostvars[node].cpus }}
{% endfor %}
```

Se a lista `nodes` tiver dois itens, o trecho é gerado duas vezes. É esse mecanismo que permite declarar cada máquina uma única vez, no inventário, e obter uma linha por nó no arquivo final.

Condicionais

A construção `{% if %}` inclui um trecho apenas quando a condição é verdadeira:

```
{% if node.gpu_device is defined %}File={{ node.gpu_device }}{% else %}Cou
```

O teste `is defined` verifica se a chave existe no item. Sem essa verificação, acessar uma chave inexistente causaria erro. Um teste muito usado é `default(valor)`, que substitui o valor ausente por um padrão:

```
{{ node.gpu_count | default(0) }}
```

O símbolo `|` aplica um **filtro** ao valor. Filtros transformam valores: `default` substitui ausentes, e `join` junta uma lista em um texto, como visto na seção 7.2.

Espaços em branco

Cada `{% ... %}` deixa uma quebra de linha no arquivo gerado. Para evitar linhas vazias, o Jinja2 aceita um traço nas bordas da marcação: `{%- ... %}` remove os espaços antes, e `{% ... -%}` remove os espaços depois. Neste capítulo, os templates usam as marcações sem traço, porque as linhas vazias não atrapalham a leitura do Slurm.

> **Referência:** Pallets. Jinja Documentation. jinja.palletsprojects.com. Sintaxe de variáveis, laços, condicionais e filtros. **Referência:** Ansible. Templating (Jinja2). docs.ansible.com. Como o Ansible usa o Jinja2 e quais variáveis estão disponíveis em um template.

7.2 `slurm.conf.j2` passo a passo

O template de `slurm.conf` fica em `roles/slurm/templates/slurm.conf.j2`. Ele combina dois tipos de conteúdo: valores fixos, que valem para qualquer cluster gerado por este tutorial, e valores vindos das variáveis, que dependem das máquinas do inventário.

Versão completa do template:

```
# Gerado pelo Ansible a partir de templates/slurm.conf.j2.
# Não editar à mão: alterações são sobrescritas na próxima execução.
ClusterName={{ cluster_name }}
SlurmctldHost={{ controller_host }}
SlurmUser=slurm
StateSaveLocation=/var/spool/slurmctld
SlurmdSpoolDir=/var/spool/slurmd
```

```
SlurmctldPidFile=/run/slurmctld.pid
SlurmdPidFile=/run/slurmd.pid
ReturnToService=2
SlurmdParameters=config_overrides

ProctrackType=proctrack/cgroup
TaskPlugin=task/cgroup,task/affinity
SchedulerType=sched/backfill
SelectType=select/cons_tres
SelectTypeParameters=CR_Core
GresTypes=gpu

AccountingStorageType=accounting_storage/none
JobAcctGatherType=jobacct_gather/none

{% for h in groups['compute'] %}
NodeName={{ h }} NodeAddr={{ hostvars[h].node_addr | default('127.0.0.1')
{% endfor %}

PartitionName=debug Nodes={{ groups['compute'] | join(',') }} Default=YES
```

Parte fixa e parte variável

As linhas sem marcação Jinja são fixas. Elas são idênticas em qualquer cluster gerado por este template. O valor de `ReturnToService`, por exemplo, foi apresentado na Parte 5 e não depende da máquina.

As linhas com marcação são variáveis:

- **`ClusterName`** recebe o nome do cluster, de `group_vars/all.yml`.
- **`SlurmctldHost`** recebe o nome do controlador, resolvido por `controller_host`. O valor precisa coincidir com o nome da máquina, como visto na Parte 5.
- **As linhas `NodeName`** são geradas uma por host do grupo `compute`. Os valores de CPU, memória e GPU vêm das variáveis de cada host.
- **A linha `PartitionName`** lista todos os nós de computação na partição `debug`.

O laço dos nós

O trecho central é este:

```
{% for h in groups['compute'] %}
NodeName={{ h }} NodeAddr={{ hostvars[h].node_addr | default('127.0.0.1')
{% endfor %}
```

O laço percorre os hosts do grupo `compute`. A variável `h` recebe o nome de cada host, e `hostvars[h]` dá acesso às variáveis desse host. Dessa forma, cada linha `NodeName` usa os valores da própria máquina.

Três detalhes merecem atenção:

- **`node_addr`** é opcional. Com uma máquina, o padrão `127.0.0.1` aponta para a própria máquina. Com várias, cada host define o endereço pelo qual o controlador o alcança.
- **`slurmd_port`** vem de `group_vars/all.yml`. Um valor compartilhado é suficiente enquanto cada nó estiver em uma máquina diferente.
- **`| default(0)`** trata o caso de um host sem a variável `gpu_count`. Sem o filtro, o template falha ao procurar um valor que não existe. Com o filtro, o host é tratado como um nó sem GPU.

A lista de partição

```
PartitionName=debug Nodes={{ groups['compute'] | join(',') }} Default=YES
```

O filtro `join(',')` junta os nomes dos hosts separados por vírgula. Com dois hosts chamados `node01` e `node02`, o resultado é `Nodes=node01,node02`.

Testando a renderização sem instalar

A renderização completa é observada com a opção `--diff`, que mostra o conteúdo que seria gravado em cada arquivo. Essa opção só produz saída útil quando a tarefa de renderização existe, e ela é escrita na Parte 8. Até lá, o comando abaixo apenas confirma que o playbook continua válido:

```
ansible-playbook site.yml --check --diff -K
```

Na Parte 8, a mesma linha de comando passa a mostrar o texto exato de `slurm.conf`, `gres.conf` e `cgroup.conf` antes de qualquer gravação. Se houver erro de sintaxe em um template, a mensagem aponta o arquivo e a linha.

> **Referência:** Ansible. ansible.builtin.template module. docs.ansible.com. Parâmetros `src`, `dest`, `mode` e a opção de renderização. **Referência:** Ansible. Magic variables, and how to access information about other hosts. docs.ansible.com. Uso de `groups` e `hostvars` dentro de templates. **Referência:** Ansible. Using filters to manipulate data. docs.ansible.com. Filtros `default`, `map` e `join`.

7.3 `gres.conf.j2` e a diferença entre nó com GPU física e nó sem GPU física

O template de `gres.conf` fica em `roles/slurm/templates/gres.conf.j2`. Ele declara os dispositivos de GPU e só deve gerar linhas para os hosts que têm GPU.

```
# Gerado pelo Ansible a partir de templates/gres.conf.j2.
{% for h in groups['compute'] if hostvars[h].gpu_count | default(0) > 0 %}
```

```
NodeName={{ h }} Name=gpu {% if hostvars[h].gpu_device is defined %}File={
{% endfor %}
```

O filtro no laço

A cláusula `if` no `for` filtra os hosts antes da iteração. Hosts sem GPU são ignorados, e o arquivo nunca contém uma linha para eles.

Dois casos de declaração

O bloco condicional dentro da linha escolhe a forma de declarar a GPU:

- **Host com dispositivo real** (variável `gpu_device` definida): a linha usa `File=`. O Slurm pode isolar o dispositivo com cgroups, como visto na Parte 6.
- **Host sem dispositivo real** (variável `gpu_device` ausente): a linha usa `Count=`. O Slurm apenas reserva unidades, sem isolar nada.

A distinção é feita pela presença ou ausência de `gpu_device` no inventário. Um exemplo de um host de computação sem dispositivo real, com duas GPUs declaradas por contagem:

```
hosts:
  <outro-hostname>:
    ansible_connection: ssh
    cpus: 2
    real_memory: 1000
    gpu_count: 2
```

Esse host recebe `Count=2`, sem linha de `File=`. A Parte 10 usa esse caso para nós sem GPU física.

> **Atenção:** o número de linhas `File=` deve bater com o `gpu_count` do host. Se `gpu_count` for 2 e `gpu_device` apontar para um único arquivo, o Slurm recebe menos dispositivos do que declarou. Confira a correspondência antes de aplicar.
>
> **Referência:** SchedMD. gres.conf: Generic Resource configuration file. slurm.schedmd.com/gres.conf.html. Sintaxe de `File=` e `Count=`.

7.4 Por que nunca editar arquivos gerados

Os arquivos `/etc/slurm/slurm.conf`, `/etc/slurm/gres.conf` e `/etc/slurm/cgroup.conf` são gerados pelo Ansible. Editar um deles diretamente parece mais rápido, mas tem três problemas:

1. **A mudança desaparece.** Na próxima execução do playbook, o template é aplicado de novo e o arquivo volta ao estado do repositório. Isso é a idempotência funcionando: o estado declarado vence o estado manual.

2. **A mudança fica fora do histórico.** Como o arquivo gerado não está no repositório, não há registro do que foi alterado nem de quando.
3. **A mudança não se repete.** Se o cluster for recriado em outra máquina, a edição manual se perde, porque ela não está descrita em nenhum arquivo versionado.

A regra prática é: altere a fonte, e não o resultado. Para mudar o número de CPUs de uma máquina, altere `cpus` no inventário. Para mudar um valor do cluster, altere `group_vars/all.yml`. Para mudar a forma como o escalonador funciona, altere o template em `roles/slurm/templates/`.

Os cabeçalhos dos templates trazem o aviso `Gerado pelo Ansible`. Ele existe para quem abrir o arquivo gerado sem saber de onde ele vem.

Para detectar edições manuais esquecidas, a execução com `--check --diff` mostra a diferença entre o que está no sistema e o que o template produziria:

```
ansible-playbook site.yml --check --diff -K
```

Se houver diferença em um arquivo gerado, a saída mostra as linhas que mudariam.

> **Referência:** Morris, K. Infrastructure as Code: Managing Servers in the Cloud. O'Reilly. Capítulo sobre configuração declarativa e o problema da deriva de configuração (configuration drift). (Verifique a edição antes de citar.)

Resumo da Parte 7

Ao final, você entende como o Jinja2 usa variáveis, laços e condicionais para gerar texto, e escreveu os três templates da role `slurm`. O `slurm.conf.j2` combina uma parte fixa com linhas geradas por nó, o `gres.conf.j2` declara apenas os nós com GPU, e a diferença entre `File=` e `Count=` depende da presença de `gpu_device`. Você também sabe por que os arquivos gerados não devem ser editados à mão.

Os templates ainda não foram aplicados ao sistema. A tarefa que os renderiza, e os handlers que reiniciam os serviços, são escritos na Parte 8.

Próxima: Parte 8, Integração do playbook ao cluster funcionando.

Parte 8: Integração do playbook ao cluster funcionando

Objetivo desta parte: escrever as tarefas e os handlers da role `slurm`, entender a ordem em que o playbook executa as roles e as tarefas, e percorrer o fluxo completo de uma mudança, da edição das variáveis até a verificação.

Pré-requisito: Partes 3 a 7 concluídas. A role `base` e a role `munge` têm suas tarefas escritas, e os três templates (`slurm.conf.j2`, `gres.conf.j2` e `cgroup.conf.j2`) existem em `roles/slurm/templates/`.

Esta parte altera o sistema. Antes de cada execução real, rode a simulação com `--check --diff`, como indicado na seção 7.2. Esta é a primeira parte em que os arquivos de configuração do Slurm são gravados em `/etc/slurm/`.

8.1 Como o playbook percorre as roles

O playbook `site.yml` lista três roles na ordem `base`, `munge` e `slurm`. Cada role contém tarefas e handlers, e o Ansible executa tudo isso em uma sequência previsível.

Uma regra do Ansible explica a sequência: por padrão, a estratégia de execução é `linear`. Ela executa a primeira tarefa em todos os hosts antes de passar para a segunda, e assim por diante. Com um único host, a diferença não aparece. Com vários hosts, ela garante que todas as máquinas concluam uma etapa antes de todas iniciarem a seguinte.

A role `slurm` tem a seguinte estrutura, já criada pela Parte 2:

```
roles/slurm/
├── tasks/main.yml        # renderiza os arquivos e garante os serviços
├── handlers/main.yml     # reinicia os serviços quando um arquivo muda
└── templates/
    ├── slurm.conf.j2
    ├── gres.conf.j2
    └── cgroup.conf.j2
```

Ao executar a role, o Ansible lê primeiro `tasks/main.yml`, de cima para baixo. Os handlers só são executados depois, no fim da role ou do playbook, e só se uma tarefa os tiver notificado.

As tarefas da role `slurm`

O arquivo `roles/slurm/tasks/main.yml` fica assim:

```
- name: Renderizar slurm.conf
  ansible.builtin.template:
    src: slurm.conf.j2
    dest: /etc/slurm/slurm.conf
    owner: root
    group: root
    mode: "0644"
  notify:
    - restart slurmctld
    - restart slurmd

- name: Renderizar gres.conf
  ansible.builtin.template:
    src: gres.conf.j2
    dest: /etc/slurm/gres.conf
    owner: root
    group: root
    mode: "0644"
  notify:
    - restart slurmctld
    - restart slurmd

- name: Renderizar cgroup.conf
  ansible.builtin.template:
    src: cgroup.conf.j2
    dest: /etc/slurm/cgroup.conf
    owner: root
    group: root
    mode: "0644"
  notify:
    - restart slurmctld
    - restart slurmd

- name: Garantir que slurmctld está ativo
  ansible.builtin.systemd:
    name: slurmctld
    state: started
    enabled: true
  when: "'controller' in group_names"

- name: Garantir que slurmd está ativo
  ansible.builtin.systemd:
    name: slurmd
    state: started
    enabled: true
  when: "'compute' in group_names"
```

Leitura:

- **`template`** lê o arquivo `.j2` da pasta `templates/`, substitui as variáveis pelos valores do inventário e de `group_vars/all.yml` e grava o

resultado em `dest`. Se o conteúdo gerado for igual ao que já existe no sistema, a tarefa reporta `ok`, e nada é gravado.
- **`notify`** avisa os handlers de que o arquivo mudou. A notificação só ocorre quando a tarefa reporta `changed`. Se o arquivo já estiver correto, nenhum handler é disparado.
- **`when: "'controller' in group_names"`** executa a tarefa apenas em hosts que pertencem ao grupo `controller`. `group_names` é uma variável especial, criada pelo Ansible, com a lista de grupos do host atual. Esse é o mecanismo que decide quais serviços cada máquina executa, a partir do inventário da Parte 3.

Note que os três arquivos de configuração são notificados aos dois handlers. Um nó de computação recebe o `slurm.conf` completo, que contém todos os nós, e não apenas a sua parte. Isso é intencional: todos os daemons precisam enxergar a mesma configuração do cluster.

> **Referência:** Ansible. ansible.builtin.template module. docs.ansible.com. Parâmetros `src`, `dest`, `owner`, `group` e `mode`.
> **Referência:** Ansible. Conditionals: group_names and when. docs.ansible.com. Uso de `group_names` para executar tarefas por grupo do inventário.

8.2 Handlers: reiniciar serviços só quando a configuração muda

Um **handler** é uma tarefa que só executa quando é notificada por outra tarefa que mudou alguma coisa. Ele evita reinícios desnecessários: um serviço que não precisa ser reiniciado continua rodando sem interrupção.

O arquivo `roles/slurm/handlers/main.yml` fica assim:

```
- name: restart slurmctld
  ansible.builtin.systemd:
    name: slurmctld
    state: restarted
  when: "'controller' in group_names"

- name: restart slurmd
  ansible.builtin.systemd:
    name: slurmd
    state: restarted
  when: "'compute' in group_names"
```

Pontos importantes:

- **O nome do handler precisa coincidir exatamente com o nome usado em `notify`.** Um erro de digitação faz o Ansible falhar com a mensagem `the requested handler could not be found`. Esse erro é fácil de cometer e fácil de corrigir: compare os nomes.

- **Cada handler tem a mesma guarda `when` das tarefas de início.** Assim, um host de computação não tenta reiniciar `slurmctld`, e um host controlador não tenta reiniciar `slurmd`. Como ambos os handlers são notificados pelas três tarefas de renderização, a guarda é o que impede a execução do serviço errado.
- **`state: restarted`** para o serviço e o inicia de novo. Se o serviço não estava rodando, ele é iniciado.

Reinício duplicado na primeira execução

Na primeira execução, as tarefas de renderização mudam os arquivos e notificam os handlers. Em seguida, a tarefa `Garantir que ... está ativo` inicia o serviço com a configuração nova. Ao final da role, os handlers são executados e reiniciam o serviço que acabou de iniciar. O resultado é um reinício a mais do que o necessário.

Esse reinício duplicado é inofensivo. O serviço termina no estado correto. Se quiser evitá-lo, o módulo `meta` permite forçar a execução dos handlers antes da tarefa de início:

```
- name: Executar handlers pendentes antes de iniciar os serviços
  ansible.builtin.meta: flush_handlers
```

Essa tarefa, colocada antes das tarefas de início, executa os handlers imediatamente. Neste tutorial, ela não é usada, porque o reinício extra não causa problema e a sequência fica mais simples de ler.

> **Referência:** Ansible. Handlers: running operations on change. docs.ansible.com. Como handlers são notificados, executados e ordenados. **Referência:** Ansible. ansible.builtin.meta module. docs.ansible.com. Ação `flush_handlers`.

8.3 Ordem de execução e dependências entre serviços

A ordem de execução importa porque cada etapa depende da anterior. Os pares abaixo mostram as dependências do projeto:

Etapa	Depende de	Motivo
`base` instala os pacotes	nada	É a base de tudo.
`munge` garante o serviço e a chave	`base`	O pacote `munge` é instalado pela role `base`. Sem ele, não há serviço nem chave.
`slurm` renderiza a configuração	`base` e `munge`	Os templates dependem dos pacotes instalados e da chave de autenticação.
`slurmctld` inicia	`slurm.conf` válido	Se a configuração tiver erro de sintaxe, o controlador não inicia.

Etapa	Depende de	Motivo
`slurmd` inicia	`munge` ativo e `slurm.conf` válido	O nó precisa autenticar as mensagens do controlador.

Ordem dentro da role `slurm`

Dentro da role, a sequência correta é: primeiro gravar a configuração, depois iniciar os serviços. As tarefas de renderização vêm antes das tarefas de início, e isso garante que o serviço só sobe com a configuração já gravada.

Se a ordem fosse invertida, o serviço subiria com uma configuração antiga ou inexistente. Ele falharia, o que atrasaria o diagnóstico.

Ordem entre hosts

Com uma única máquina, todas as tarefas acontecem na mesma máquina, na ordem da role. Com várias máquinas, o controlador deve estar pronto antes dos nós de computação se registrarem. O Ansible não garante essa relação apenas pela ordem das roles, porque as tarefas de cada etapa rodam em paralelo nos hosts.

A solução usual é colocar o controlador em um play separado, executado antes do play dos nós. Como esta apostila usa um único playbook, o efeito é atenuado pela opção `ReturnToService=2` (Parte 5): um nó que chega antes do controlador fica indisponível até conseguir se registrar. Esse comportamento é esperado e não exige intervenção. A divisão em plays é tratada na Parte 10.

> **Referência:** Ansible. Controlling playbook execution: strategies. docs.ansible.com. Estratégia `linear` e o comportamento de execução entre hosts. **Referência:** SchedMD. slurm.conf: ReturnToService. slurm.schedmd.com/slurm.conf.html. Comportamento de retorno de nós ao serviço.

8.4 Fluxo completo de uma mudança de configuração

Este é o procedimento que se repete sempre que a configuração muda. Ele é o mesmo para uma mudança pequena, como ajustar a memória de um nó, e para uma grande, como adicionar um nó.

Passo 1: editar a fonte

Altere `inventory/hosts.yml`, e não os arquivos gerados. Para reduzir a memória declarada de uma máquina, altere `real_memory` no grupo `compute`:

```
compute:
  hosts:
```

```
        <hostname>:
          ansible_connection: local
          cpus: 8
          real_memory: 7600
          gpu_count: 1
          gpu_device: /dev/nvidia0
```

Passo 2: validar a sintaxe

```
ansible-playbook site.yml --syntax-check
```

Erros de indentação em YAML são detectados aqui.

Passo 3: simular

```
ansible-playbook site.yml --check --diff -K
```

A saída mostra, para cada arquivo que mudaria, as linhas acrescentadas e removidas. Leia com atenção antes de seguir. Na simulação, os handlers não são executados, então nenhum serviço é reiniciado.

Passo 4: aplicar

```
ansible-playbook site.yml -K
```

Ao final, o Ansible mostra um resumo chamado `PLAY RECAP`, com a contagem de tarefas `ok`, `changed`, `skipped` e `failed` para cada host. Uma execução sem falhas tem `failed=0`.

Passo 5: verificar a idempotência

Execute o playbook de novo, sem nenhuma mudança:

```
ansible-playbook site.yml -K
```

Na segunda execução, o `PLAY RECAP` deve mostrar `changed=0`, ou no máximo a tarefa de atualização de cache de pacotes. Se houver outras tarefas marcadas como `changed`, algum estado não é estável, e a causa deve ser investigada antes de seguir.

Passo 6: registrar a mudança

Depois de validar, registre a mudança no controle de versão. Se o projeto usa Git, por exemplo:

```
git add inventory/hosts.yml group_vars/all.yml roles/slurm/templates/
git commit -m "Declara novo nó no cluster"
```

A validação de que o cluster funciona, com `sinfo` e jobs de teste, é feita na Parte 9.

Por que o nome do nó vem do inventário

Com a abordagem B da Parte 3, o nome de cada nó é o nome do host no inventário, e a lista de nós é lida de `groups['compute']`. Não há nome copiado em outro arquivo, então adicionar uma máquina exige apenas uma entrada no inventário. Os templates da Parte 7 passam a valer para uma máquina ou para várias, sem nenhuma alteração.

> **Referência:** Ansible. Using variables: variable precedence and scope. docs.ansible.com. Precedência entre fontes de variáveis, incluindo `group_vars` e variáveis de host.

Resumo da Parte 8

Ao final, a role `slurm` está completa: renderiza os três arquivos de configuração, notifica os handlers quando eles mudam, e inicia o `slurmctld` ou o `slurmd` conforme o grupo do host. Você entende a ordem de execução entre as roles e dentro da role, a função dos handlers, e o fluxo de seis passos para qualquer mudança: editar a fonte, validar, simular, aplicar, verificar a idempotência e registrar.

Você também sabe por que a topologia vem do inventário: a lista de nós é derivada dos grupos, e as variáveis de cada máquina ficam com a própria máquina, de modo que adicionar uma máquina não exige editar mais de um arquivo.

A configuração está aplicada, mas ainda não foi verificada pelo próprio Slurm. Essa verificação é feita na Parte 9.

Próxima: Parte 9, Validação do cluster.

Parte 9: Validação do cluster

Objetivo desta parte: confirmar, com os comandos do próprio Slurm, que o controlador enxerga os nós, que os nós estão disponíveis, que um job é aceito e executado, e que o job recebe apenas as GPUs que pediu.

Pré-requisito: Partes 3 a 8 concluídas e aplicadas. O playbook termina com `failed=0`, e a segunda execução mostra `changed=0`.

Esta parte não altera a configuração. Os comandos consultam o estado do cluster e enviam jobs de teste. Os jobs de teste são curtos e não modificam o sistema.

9.1 `sinfo`, `scontrol show node` e `squeue`

Três comandos mostram o estado do cluster, cada um com um foco diferente:

Comando	Pergunta que responde
`sinfo`	Quais partições e nós existem, e em qual estado estão?
`scontrol show node <nome>`	Quais recursos esse nó declara, e por que ele está no estado atual?
`squeue`	Quais jobs estão na fila ou em execução?

`sinfo`: visão geral

```
sinfo
```

Saída ilustrativa, com um nó:

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*       up   infinite      1   idle node01
```

Leitura:

- **`PARTITION`** é o nome da partição. O asterisco indica a partição padrão.
- **`AVAIL`** mostra se a partição aceita jobs. `up` significa que aceita.
- **`NODES`** é a quantidade de nós nesse estado.
- **`STATE`** é o estado dos nós. O estado esperado é `idle`, que significa disponível e sem jobs.
- **`NODELIST`** lista os nós.

Para ver um nó por linha, com mais detalhes:

```
sinfo -N -l
```

Estados mais comuns

Estado	Significado
`idle`	Disponível e sem jobs. É o estado desejado após a configuração.
`alloc`	Totalmente alocado a jobs.
`mix`	Parcialmente alocado. Parte dos recursos está livre.
`down`	Indisponível. O controlador não consegue se comunicar com o `slurmd`, ou a configuração declarada não corresponde ao hardware.
`drain`	Retirado de uso pelo administrador. Não recebe novos jobs, mas os atuais continuam.
`unk` ou `UNKNOWN`	Estado inicial. O controlador ainda não recebeu resposta do nó.

Se o nó aparecer como `down` ou `unk` logo após a aplicação, espere alguns segundos e consulte de novo. O `slurmd` pode ainda estar se registrando. Se o estado persistir, o motivo está registrado no próprio nó, como mostrado a seguir.

`scontrol show node`: detalhes de um nó

```
scontrol show node node01
```

Saída ilustrativa, resumida:

```
NodeName=node01 Arch=x86_64 CoresPerSocket=4
   CPUAlloc=0 CPUTot=8 CPULoad=0.02
   Gres=gpu:1
   RealMemory=7800 AllocMem=0
   State=IDLE ThreadsPerCore=2
   Reason=Not responding [slurm@2026-01-01T10:00:00]
```

Os campos que importam nesta etapa são:

- **`CPUTot`** deve corresponder ao valor declarado em `cpus` (Parte 3).
- **`RealMemory`** deve corresponder ao valor declarado em `real_memory`.
- **`Gres`** deve mostrar `gpu:1` se o nó tem uma GPU declarada (Parte 6).
- **`State`** deve ser `IDLE`.
- **`Reason`** aparece apenas quando o nó está em estado anômalo, e explica o motivo.

Se `CPUTot` ou `RealMemory` não baterem com o que foi declarado, o Slurm ignorou a declaração ou o nó foi rejeitado. Verifique o valor de `slurmd -C` (Parte 4) e a configuração gerada.

Quando um nó está em `DOWN` ou `DRAIN` por um motivo já resolvido, o administrador pode devolvê-lo ao serviço. Esse comando altera o estado e exige privilégio:

```
sudo scontrol update nodename=node01 state=resume
```

Use esse comando apenas depois de entender por que o nó saiu de serviço. Devolver um nó com problema real faz jobs falharem.

squeue: a fila

`squeue`

Saída ilustrativa, sem jobs:

```
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELI
```

Um cabeçalho sem linhas abaixo significa fila vazia, que é o estado esperado neste ponto. Os campos mais importantes são `ST` (o estado do job, como `PD` para pendente e `R` para em execução) e `NODELIST(REASON)`, que mostra o nó onde o job roda ou o motivo de estar parado.

> **Referência:** SchedMD. sinfo: View information about Slurm nodes and partitions. slurm.schedmd.com/sinfo.html. Campos de saída e estados de nós. **Referência:** SchedMD. scontrol: View or modify Slurm configuration and state. slurm.schedmd.com/scontrol.html. Comandos `show node` e `update`. **Referência:** SchedMD. squeue: View information about jobs in the scheduling queue. slurm.schedmd.com/squeue.html. Campos e estados de jobs.

9.2 Primeiro job: `srun` e `sbatch`

O Slurm oferece duas formas de executar um job, com comportamentos diferentes.

Comando	Comportamento	Quando usar
`srun`	Executa imediatamente, bloqueia o terminal até terminar, e mostra a saída na tela.	Testes rápidos e comandos curtos.
`sbatch`	Coloca o job na fila e devolve o controle ao terminal. A saída vai para um arquivo.	Trabalhos que podem demorar, ou que devem sobreviver ao fechamento do terminal.

Teste com `srun`

`srun hostname`

Saída esperada: o nome da máquina, impresso no terminal. Se o comando ficar esperando indefinidamente, verifique com `squeue` se o job está pendente e o motivo. Um job pendente com motivo `Resources` significa que não há recursos livres. Um motivo como `ReqNodeNotAvail` indica que o nó não está disponível, e o estado do nó deve ser investigado com `scontrol show node`.

Teste com `sbatch`

Crie o arquivo `teste-basico.sh`:

```
#!/bin/bash
#SBATCH --job-name=teste-basico
#SBATCH --output=saida_%j.log

echo "Executando em: $(hostname)"
echo "Data: $(date)"
```

Envie o job:

```
sbatch teste-basico.sh
```

A saída do comando mostra o número do job:

```
Submitted batch job 1
```

O número varia. Acompanhe com `squeue` até o job sumir da fila, o que indica término. Depois, leia o arquivo de saída, substituindo `<numero>` pelo número mostrado:

```
cat saida_<numero>.log
```

O arquivo deve conter o nome da máquina e a data. Se o job terminou sem gerar o arquivo, o problema está quase sempre no caminho do `--output` ou em permissões do diretório onde o `sbatch` foi executado.

Cancelando um job

Um job pode ser cancelado enquanto está na fila ou em execução:

```
scancel <numero>
```

Use `squeue` antes para confirmar o número. O cancelamento de um job não afeta os outros.

> **Referência:** SchedMD. srun: Run parallel jobs. slurm.schedmd.com/srun.html. Execução interativa e bloqueante. **Referência:** SchedMD. sbatch: Submit a batch script to Slurm. slurm.schedmd.com/sbatch.html. Diretivas `#SBATCH` e o arquivo de saída. **Referência:** SchedMD. scancel: Signal or cancel jobs, job arrays, or job steps. slurm.schedmd.com/scancel.html.

9.3 Verificando a GPU dentro de um job

Até aqui, o Slurm confirmou que o nó existe e que o job executa. Falta confirmar o ponto central da configuração: o job recebe a GPU que pediu, e somente ela.

Teste positivo: o job vê a GPU pedida

```
srun --gres=gpu:1 nvidia-smi -L
```

Saída esperada, ilustrativa:

```
GPU 0: NVIDIA ... (UUID: GPU-...)
```

O comando lista uma GPU. Se a máquina tiver mais de uma, o job recebe apenas a quantidade pedida em `--gres`.

Para placas de outros fabricantes, substitua a ferramenta pela equivalente (`rocm-smi` para AMD). A ideia do teste é a mesma: a ferramenta do fabricante, executada dentro do job, lista as GPUs visíveis para ele.

Teste negativo: o job sem GPU não a enxerga

Este é o teste que demonstra o isolamento. Execute o mesmo comando sem pedir GPU:

```
srun nvidia-smi -L
```

Com `ConstrainDevices=yes` (Parte 6), o job não tem acesso ao dispositivo, e a ferramenta deve falhar ou reportar que não encontrou GPUs. A mensagem exata depende da versão do driver, mas o resultado esperado é o mesmo: nenhuma GPU listada.

Se o comando sem `--gres` listar a GPU normalmente, o isolamento não está funcionando. Os motivos mais comuns são:

- o nó foi declarado com `Count` em vez de `File` (Parte 6, seção 6.3), o que impede o isolamento de dispositivo;
- `cgroup.conf` não contém `ConstrainDevices=yes`;
- o `slurmd` não foi reiniciado depois da mudança de `cgroup.conf`.

Verifique cada um, na ordem, antes de considerar o problema resolvido.

Teste de consistência: o que o Slurm declarou

Para confirmar que o Slurm alocou a GPU que o job pediu, consulte o job enquanto ele está em execução. Use um job que dure alguns segundos:

```
sbatch --gres=gpu:1 --wrap="sleep 30"
squeue
scontrol show job <numero>
```

A linha `Gres=gpu:1` no campo de alocação mostra que o job recebeu uma GPU. Essa informação vem do controlador, e complementa o teste com `nvidia-smi`, que vem da própria máquina.

A opção `--wrap` permite enviar um comando direto, sem criar um arquivo de script. É útil para testes rápidos com `sbatch`.

Histórico de jobs

Para consultar jobs já terminados, o comando `sacct` é usado. Nesta etapa, o histórico ainda não está ativo, porque o armazenamento de contabilidade foi desligado na configuração. Se você executar:

```
sacct
```

a mensagem indicará que o armazenamento de histórico está desativado. Esse comportamento é esperado. O histórico é habilitado na Parte 10.

> **Referência:** NVIDIA. nvidia-smi: NVIDIA System Management Interface. Guia do usuário, docs.nvidia.com. Opção `-L` para listar dispositivos. **Referência:** SchedMD. Generic Resource (GRES) Scheduling. slurm.schedmd.com/gres.html. Como o Slurm aloca recursos genéricos a jobs.

Resumo da Parte 9

Ao final, você confirmou quatro pontos com o próprio Slurm: o controlador enxerga o nó, o nó está em `idle` com os recursos declarados, um job executa com `srun` e com `sbatch`, e o job recebe apenas as GPUs que pediu. O teste negativo, sem `--gres`, é a prova de que o isolamento por cgroups funciona.

Se algum teste falhar, a Parte 11 reúne os sintomas mais comuns e a forma de diagnosticar cada um.

Próxima: Parte 10, Expansão.

Parte 10: Expansão

Objetivo desta parte: levar o cluster além de uma máquina: declarar nós virtuais no mesmo host, distribuir a configuração de autenticação entre várias máquinas e ativar o histórico de jobs.

Pré-requisito: Partes 3 a 9 concluídas. O cluster de uma máquina valida com `sinfo` e com um job de `srun` e de `sbatch`. A topologia vem do inventário, como descrito na Parte 3 (abordagem B).

Esta parte altera o sistema e pode envolver mais de uma máquina. Use a simulação com `--check --diff` antes de cada execução. Cada seção pode ser aplicada de forma independente, nesta ordem: 10.1 não depende de outras máquinas, 10.2 exige pelo menos duas, e 10.3 pode ser feito a qualquer momento depois da Parte 9.

10.1 Nós simulados no mesmo host e o diretório de spool

Às vezes é útil ter mais nós do que máquinas. Por exemplo, para testar filas, partições e a lógica de reserva de recursos sem comprar hardware. Um **nó simulado** (ou **nó virtual**) é um processo `slurmd` adicional, rodando no mesmo host, que se apresenta ao controlador como se fosse uma máquina separada.

Um nó simulado tem limites que precisam ser entendidos antes do uso:

- **Não tem hardware próprio.** As GPUs declaradas com `Count` são unidades reservadas, não dispositivos reais. Programas CUDA não rodam nesses nós.
- **Não isola recursos reais.** O isolamento por cgroups (Parte 6) funciona apenas para dispositivos que existem. Dois nós simulados no mesmo host compartilham a memória e o processador reais da máquina.
- **Serve para testar o escalonador.** O uso correto é verificar filas, partições, pedidos de recursos e comportamento de estados. Medições de desempenho feitas nesses nós não são válidas.

Declarando os nós virtuais

Os nós virtuais pertencem ao host que os executa. Por isso, a lista deles fica como variável do host, no inventário:

```
compute:
  hosts:
    <hostname>:
      ansible_connection: local
      cpus: 8
      real_memory: 7600
```

```
    gpu_count: 1
    gpu_device: /dev/nvidia0
    virtual_nodes:
      - name: vnode1
        port: 6819
        cpus: 2
        real_memory: 1500
        gpu_count: 2
      - name: vnode2
        port: 6820
        cpus: 2
        real_memory: 1500
        gpu_count: 0
```

Cada item descreve um nó simulado com nome, porta, CPUs, memória e GPUs. As portas precisam ser diferentes entre si e diferentes da porta do nó real, que vem de `slurmd_port`. A soma de memória dos nós simulados não deve passar da memória física da máquina. O Slurm não verifica isso, e o resultado seria lentidão e falhas por falta de memória.

Atualizando o template de `slurm.conf`

O laço de nós passa a ter dois níveis: primeiro o nó real de cada host, depois os virtuais desse host. A lista de partição precisa incluir os dois tipos. O bloco abaixo substitui os trechos de nós e de partição no `slurm.conf.j2` da Parte 7:

```
{% for h in groups['compute'] %}
NodeName={{ h }} NodeAddr={{ hostvars[h].node_addr | default('127.0.0.1')
{% for v in hostvars[h].virtual_nodes | default([]) %}
NodeName={{ v.name }} NodeAddr=127.0.0.1 Port={{ v.port }} CPUs={{ v.cpus
{% endfor %}
{% endfor %}

{% set ns = namespace(nomes=[]) %}
{% for h in groups['compute'] %}
{% set ns.nomes = ns.nomes + [h] %}
{% for v in hostvars[h].virtual_nodes | default([]) %}
{% set ns.nomes = ns.nomes + [v.name] %}
{% endfor %}
{% endfor %}
PartitionName=debug Nodes={{ ns.nomes | join(',') }} Default=YES MaxTime=I
```

Pontos a entender:

- **O laço interno** percorre `virtual_nodes` do host atual. O filtro `default([])` garante que um host sem nós virtuais produza zero linhas, sem erro.
- **`NodeAddr=127.0.0.1`** é fixo, porque os nós simulados rodam na própria máquina.
- **`namespace`** é um recurso do Jinja2 que permite alterar uma variável dentro de um laço. Sem ele, cada `set` dentro do `for` criaria uma

variável local que some ao fim do laço. Ele é o único jeito de montar a lista de nomes em duas etapas.

Atualizando o template de `gres.conf`

Os nós virtuais com GPUs recebem `Count`, porque não têm dispositivo próprio. O nó real continua recebendo `File`:

```
# Gerado pelo Ansible a partir de templates/gres.conf.j2.
{% for h in groups['compute'] %}
{% if hostvars[h].gpu_count | default(0) > 0 %}
NodeName={{ h }} Name=gpu {% if hostvars[h].gpu_device is defined %}File={
{% endif %}
{% for v in hostvars[h].virtual_nodes | default([]) if v.gpu_count | defau
NodeName={{ v.name }} Name=gpu Count={{ v.gpu_count }}
{% endfor %}
{% endfor %}
```

Uma unidade do systemd para cada nó virtual

Cada nó virtual precisa de um processo próprio, iniciado com a opção `-N`, que dá ao `slurmd` o nome do nó que ele representa. A forma padrão de manter um processo ativo no Linux é uma unidade do systemd. O template abaixo gera uma unidade por nó virtual, e fica em `roles/slurm/templates/slurmd-virtual.service.j2`:

```
# Gerado pelo Ansible a partir de templates/slurmd-virtual.service.j2.
[Unit]
Description=Nó virtual {{ item.name }} do Slurm
After=network-online.target munge.service
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/sbin/slurmd -D -N {{ item.name }}
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Leitura:

- **`-D`** mantém o `slurmd` em primeiro plano, que é o modo esperado por uma unidade do tipo `simple`.
- **`-N`** informa o nome do nó que esse processo representa.
- **`Restart=on-failure`** reinicia o processo se ele cair por erro.
- **`After=munge.service`** garante que a autenticação esteja disponível antes do início.

As tarefas que renderizam e reiniciam essas unidades são adicionadas ao final de `roles/slurm/tasks/main.yml`:

```
- name: Renderizar unidades dos nós virtuais
  ansible.builtin.template:
    src: slurmd-virtual.service.j2
    dest: "/etc/systemd/system/slurmd-{{ item.name }}.service"
    owner: root
    group: root
    mode: "0644"
  loop: "{{ virtual_nodes | default([]) }}"
  register: unidades_virtuais

- name: Reiniciar as unidades que mudaram
  ansible.builtin.systemd:
    name: "slurmd-{{ item.item.name }}"
    state: restarted
    enabled: true
    daemon_reload: true
  loop: "{{ unidades_virtuais.results }}"
  when: item.changed
```

A primeira tarefa grava uma unidade por nó virtual e guarda o resultado de cada uma em `unidades_virtuais`. A segunda reinicia somente as unidades que mudaram. `daemon_reload: true` pede ao systemd que releia as unidades antes de reiniciar, o que é necessário quando um arquivo de unidade muda.

O diretório de spool

Cada `slurmd` grava arquivos temporários em um diretório de spool, configurado por `SlurmdSpoolDir`. Esse valor é único no `slurm.conf`, e todas as instâncias do `slurmd` leem o mesmo arquivo. Se todos os nós virtuais usarem o mesmo diretório, eles disputarão os mesmos arquivos, e o comportamento fica imprevisível.

Há duas saídas:

1. **Caminho de spool distinto por instância.** Algumas versões do Slurm aceitam, na opção `SlurmdSpoolDir`, uma substituição que gera um caminho diferente para cada nó. Antes de usar essa forma, confirme na documentação da versão instalada (`man slurm.conf`) se a sintaxe é suportada. Esta apostila não assume essa sintaxe, porque ela depende da versão.
2. **Nós virtuais em máquinas separadas.** Se a sintaxe não estiver disponível, cada nó simulado pode rodar em uma máquina virtual própria, como descrito na seção 10.2. Esse caminho não tem o problema do spool compartilhado, porque cada máquina tem o próprio sistema de arquivos.

Antes de escolher, verifique qual opção a sua versão suporta. Escolher a primeira sem confirmar leva a um cluster que parece funcionar, mas falha em jobs concorrentes.

Verificando os nós virtuais

```
ansible-playbook site.yml --check --diff -K
ansible-playbook site.yml -K
sinfo -N -l
```

O `sinfo -N -l` deve listar o nó real e os virtuais, cada um com seu estado. Para um nó específico, confira os recursos declarados:

```
scontrol show node vnode1
```

Para verificar que um job chega ao nó virtual, use a opção `-w`, que escolhe o nó:

```
srun -w vnode1 hostname
```

A saída deve ser o nome do host real, porque o `slurmd` virtual roda na mesma máquina. Isso é esperado. O que muda é o nó que o Slurm reporta como alocado, que aparece em `squeue` durante a execução.

> **Referência:** SchedMD. slurmd: Slurm compute node daemon. slurm.schedmd.com. Descrição da opção `-N`, usada para representar nós simulados. **Referência:** Jinja Documentation. Assignments: namespace objects. jinja.palletsprojects.com. Uso de `namespace` para alterar variáveis dentro de laços. **Referência:** systemd. systemd.service(5). freedesktop.org. Tipos de serviço, `Restart` e `ExecStart`.

10.2 Múltiplas máquinas virtuais e distribuição da chave munge

Com várias máquinas, o cluster deixa de ser simulado e passa a ter topologia de rede real. Nesta seção, as máquinas são máquinas virtuais criadas com o Multipass, sem GPU. Elas servem para validar a orquestração, a comunicação entre nós e a autenticação. O mesmo procedimento vale para máquinas físicas, desde que elas sejam alcançáveis por SSH.

Preparando o acesso por SSH

O Ansible acessa cada máquina por SSH. Para isso, é preciso uma chave, criada uma vez na máquina de controle:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_cluster -N ""
```

A chave pública é colocada nas máquinas virtuais durante a criação, por meio de um arquivo de inicialização. Crie `cloud-init.yaml` com o conteúdo da chave pública:

```
#cloud-config
ssh_authorized_keys:
  - <conteúdo de ~/.ssh/id_ed25519_cluster.pub>
```

Crie as máquinas com esse arquivo:

```
multipass launch --name ctl --cpus 2 --memory 2G --cloud-init cloud-init.y
multipass launch --name node01 --cpus 2 --memory 2G --cloud-init cloud-ini
multipass list
```

A saída de `multipass list` mostra o endereço de cada máquina. Esses endereços vão para o inventário.

Inventário com várias máquinas

Com máquinas separadas, cada host recebe o endereço de conexão com `ansible_host`, e o acesso é feito por SSH:

```
all:
  children:
    controller:
      hosts:
        ctl:
          ansible_host: <ip-do-ctl>
    compute:
      hosts:
        node01:
          ansible_host: <ip-do-node01>
          cpus: 2
          real_memory: 1500
          gpu_count: 0
  vars:
    ansible_user: ubuntu
    ansible_ssh_private_key_file: ~/.ssh/id_ed25519_cluster
```

Pontos a notar:

- **O nome de cada host é o hostname da máquina.** No Multipass, o hostname é o próprio nome dado no `launch`. Essa regra mantém a abordagem B da Parte 3: o nome no inventário é o nome do nó.
- **`gpu_count: 0`** faz o template não gerar nenhuma linha `Gres` para o host, como visto na Parte 7.
- **`vars`** no nível do grupo `all` aplica a conexão SSH a todos os hosts.
- **`controller_host`** continua sendo `groups['controller'][0]`, que aqui resolve para `ctl`.

Resolução de nomes entre as máquinas

Os nós precisam se encontrar pelo nome. Quando o `node01` procurar o `ctl`, o nome precisa resolver para o endereço certo. A forma mais simples, para um cluster pequeno, é registrar os hosts do inventário no arquivo `/etc/hosts` de cada máquina. Essa tarefa vai à role `base`:

```
- name: Registrar os hosts do inventário em /etc/hosts
  ansible.builtin.lineinfile:
    path: /etc/hosts
    regexp: "^.*\\s{{ item }}$"
    line: "{{ hostvars[item].ansible_host }} {{ item }}"
  loop: "{{ groups['all'] }}"
  when: hostvars[item].ansible_host is defined
```

A tarefa percorre todos os hosts do inventário e grava uma linha por host no `/etc/hosts`. `lineinfile` com `regexp` substitui a linha existente em vez de duplicá-la, o que mantém a tarefa idempotente. A condição `when` pula hosts sem `ansible_host`, como a própria máquina de controle quando ela é conectada localmente.

Distribuindo a chave do munge

A chave do munge precisa ser a mesma em todas as máquinas (Parte 4). A chave é gerada no controlador, e cada nó de computação recebe uma cópia. A distribuição usa três passos, executados na role `munge`:

1. O controlador copia a chave para a máquina de controle do Ansible, em uma pasta de segredos.
2. Cada nó de computação recebe a chave dessa pasta.
3. A pasta temporária é apagada ao final.

Antes da role, crie o arquivo de handlers, que ainda não existe. Ele tem a mesma forma dos outros:

```
# roles/munge/handlers/main.yml
- name: restart munge
  ansible.builtin.systemd:
    name: munge
    state: restarted
```

As tarefas da role `munge` ficam assim:

```
- name: Criar a pasta local de segredos
  ansible.builtin.file:
    path: "{{ playbook_dir }}/.segredos"
    state: directory
    mode: "0700"
  delegate_to: localhost
  run_once: true

- name: Copiar a chave do munge do controlador para a máquina de controle
  ansible.builtin.fetch:
    src: /etc/munge/munge.key
    dest: "{{ playbook_dir }}/.segredos/munge.key"
    flat: true
  when: inventory_hostname == groups['controller'][0]
  no_log: true
```

```
- name: Distribuir a chave do munge para os nós de computação
  ansible.builtin.copy:
    src: "{{ playbook_dir }}/.segredos/munge.key"
    dest: /etc/munge/munge.key
    owner: munge
    group: munge
    mode: "0400"
  when: "'compute' in group_names"
  no_log: true
  notify: restart munge

- name: Apagar a cópia local da chave
  ansible.builtin.file:
    path: "{{ playbook_dir }}/.segredos/munge.key"
    state: absent
  delegate_to: localhost
  run_once: true
  no_log: true
```

Leitura dos pontos que exigem atenção:

- **`fetch` e `copy` com `src`** são usados em vez de `content`, porque a chave é um arquivo binário. Um texto convertido para UTF-8 corromperia os bytes.
- **`no_log: true`** impede que o conteúdo das tarefas apareça na saída do Ansible. Sem isso, partes da chave ficariam no log do terminal.
- **A pasta `.segredos`** deve ficar fora do controle de versão. Se o projeto usar Git, acrescente essa pasta ao `.gitignore`.
- **`delegate_to: localhost`** executa a tarefa na máquina de controle, e não no host remoto.
- **A tarefa de cópia só roda nos nós de computação.** No controlador, a chave já está correta, porque foi gerada ali.

Esta abordagem é didática. Ela deixa a chave, por um instante, no disco da máquina de controle. Em ambientes que exigem mais proteção, a chave deve ficar em um cofre de segredos, como o Ansible Vault descrito na seção 10.3. A alternativa mais segura não está descrita nesta apostila.

Verificação

A chave deve ser idêntica nas máquinas. Compare a soma de verificação em cada uma:

```
ansible all -b -m ansible.builtin.command -a "md5sum /etc/munge/munge.key"
```

Os valores devem ser iguais. A autenticação entre máquinas é testada com o munge, enviando uma credencial por SSH:

```
munge -n | ssh node01 unmunge
```

A saída deve mostrar `STATUS: Success (0)`. Se mostrar erro de credencial, as chaves são diferentes, ou os relógios das máquinas estão muito distantes.

O munge rejeita credenciais com diferença de tempo acima do limite configurado. Mantenha as máquinas com sincronização de horário (NTP), que é o padrão em Ubuntu.

Por fim, confirme que o controlador enxerga o nó de computação:

```
sinfo -N -l
```

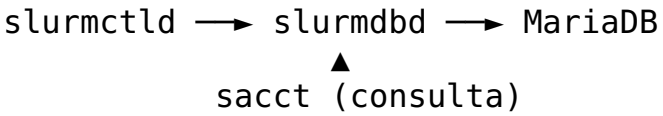
O `node01` deve aparecer em `idle`. Se aparecer `down`, verifique a conectividade nas portas padrão do Slurm, que são 6817 para o controlador e 6818 para os nós, e o motivo registrado em `scontrol show node node01`.

> **Referência:** Multipass. Multipass documentation: cloud-init. multipass.run. Uso de `--cloud-init` para configurar chaves e usuários na criação da máquina. **Referência:** Ansible. ansible.builtin.fetch module. docs.ansible.com. Parâmetro `flat` e cópia de arquivos do host remoto para a máquina de controle. **Referência:** Ansible. Keeping secrets: no_log and Ansible Vault. docs.ansible.com. Uso de `no_log` e cofres de segredos. **Referência:** Dun, C. MUNGE Uid 'N' Gid Emporium. dun.github.io/munge. Requisitos de sincronização de relógio e formato de credenciais.

10.3 Accounting com slurmdbd e MariaDB

Até aqui, o Slurm registra apenas o estado atual. O **accounting** (contabilidade) guarda o histórico de cada job: quem o enviou, quando começou, quanto tempo durou e em que estado terminou. Esse histórico é o que permite consultas como `sacct` e análises de uso do cluster.

A arquitetura tem três partes, como vista na Parte 1:



- **MariaDB** é o banco de dados que armazena os registros.
- **slurmdbd** é o daemon que recebe os registros do controlador e os grava no banco.
- **sacct** consulta os registros por meio do slurmdbd.

Nesta seção, o slurmdbd roda no controlador. Em clusters maiores, ele pode rodar em outra máquina.

Pacotes

Instale o banco e o daemon de accounting. Eles são pacotes separados do `slurm-wlm`:

```
- name: Instalar MariaDB e slurmdbd
  ansible.builtin.apt:
```

```
    name:
      - mariadb-server
      - slurmdbd
    state: present
    update_cache: true
  when: "'controller' in group_names"
```

Confirme os nomes na sua versão com `apt policy slurmdbd`, como visto na Parte 2.

Banco e usuário

O Ansible tem módulos próprios para criar bancos e usuários no MariaDB. Eles fazem parte da coleção `community.mysql`, instalada com:

```
ansible-galaxy collection install community.mysql
```

As tarefas usam a senha do usuário do banco, que deve ser tratada como segredo. O Ansible Vault guarda valores sensíveis cifrados dentro do repositório. Para gerar o valor cifrado:

```
ansible-vault encrypt_string 'senha-do-banco' --name slurm_db_password
```

O resultado é um bloco cifrado, que pode ser colado em `group_vars/all.yml`. Ele só pode ser lido com a senha do cofre. Guarde essa senha fora do repositório.

As tarefas de banco ficam em uma role própria, `slurmdbd`, criada nesta parte. Ela tem a mesma estrutura das outras roles (tarefas, handlers e templates), e precisa ser incluída no playbook antes da role `slurm`, porque o slurmdbd deve estar pronto antes do controlador:

```
- name: Configuração mínima do cluster
  hosts: all
  become: true
  roles:
    - base
    - munge
    - slurmdbd
    - slurm
```

Dentro da role `slurmdbd`, o handler que reinicia o serviço segue o mesmo padrão dos demais:

```
# roles/slurmdbd/handlers/main.yml
- name: restart slurmdbd
  ansible.builtin.systemd:
    name: slurmdbd
    state: restarted
  when: "'controller' in group_names"
```

As tarefas de banco são:

```
- name: Criar o banco do histórico
  community.mysql.mysql_db:
    name: slurm_acct_db
    state: present
    login_unix_socket: /run/mysqld/mysqld.sock
  when: "'controller' in group_names"

- name: Criar o usuário do histórico
  community.mysql.mysql_user:
    name: slurm
    password: "{{ slurm_db_password }}"
    priv: "slurm_acct_db.*:ALL"
    host: localhost
    state: present
    login_unix_socket: /run/mysqld/mysqld.sock
  no_log: true
  when: "'controller' in group_names"
```

Os dois módulos são idempotentes: criar um banco ou um usuário que já existe não causa alteração.

Configuração do slurmdbd

O slurmdbd lê o arquivo `/etc/slurm/slurmdbd.conf`. Ele contém a senha do banco, por isso tem permissões restritas. O template fica em `roles/slurmdbd/templates/slurmdbd.conf.j2`:

```
# Gerado pelo Ansible a partir de templates/slurmdbd.conf.j2.
AuthType=auth/munge
DbdHost={{ controller_host }}
SlurmUser=slurm
LogFile=/var/log/slurm/slurmdbd.log
PidFile=/run/slurmdbd.pid
StorageType=accounting_storage/mysql
StorageHost=localhost
StorageUser=slurm
StoragePass={{ slurm_db_password }}
StorageLoc=slurm_acct_db
```

Leitura:

- **`DbdHost`** informa em qual máquina o slurmdbd roda.
- **`StorageType=accounting_storage/mysql`** indica o banco usado.
- **`StorageLoc`** é o nome do banco criado acima.
- **`StoragePass`** recebe a senha, que o Ansible decifra na execução.

O arquivo precisa de permissões restritas, e o Slurm exige que apenas o usuário `slurm` consiga lê-lo:

```
- name: Renderizar slurmdbd.conf
  ansible.builtin.template:
    src: slurmdbd.conf.j2
```

```
    dest: /etc/slurm/slurmdbd.conf
    owner: slurm
    group: slurm
    mode: "0600"
  no_log: true
  when: "'controller' in group_names"
  notify: restart slurmdbd
```

A pasta de logs também precisa existir, com dono `slurm`:

```
- name: Criar a pasta de logs do slurmdbd
  ansible.builtin.file:
    path: /var/log/slurm
    state: directory
    owner: slurm
    group: slurm
    mode: "0755"
  when: "'controller' in group_names"
```

Mudanças no `slurm.conf`

O controlador precisa saber que o histórico está ativo. A forma mais simples é uma variável que liga e desliga o accounting, em `group_vars/all.yml`:

```
slurm_accounting: true
```

O template de `slurm.conf` escolhe a configuração conforme essa variável. O bloco abaixo substitui as linhas fixas de accounting da Parte 7:

```
{% if slurm_accounting | default(false) %}
AccountingStorageType=accounting_storage/slurmdbd
AccountingStorageHost={{ controller_host }}
JobAcctGatherType=jobacct_gather/linux
{% else %}
AccountingStorageType=accounting_storage/none
JobAcctGatherType=jobacct_gather/none
{% endif %}
```

O filtro `default(false)` mantém o comportamento anterior quando a variável não existe. Assim, o cluster sem accounting continua válido.

Ordem de inicialização

O slurmdbd precisa estar ativo antes do controlador, porque o controlador tenta conectar-se a ele no início. Por isso, a última tarefa da role `slurmdbd` garante que o serviço esteja iniciado. Como a role `slurmdbd` vem antes de `slurm` no playbook, o `slurmctld` só é iniciado depois que o histórico já está disponível:

```
- name: Garantir que slurmdbd está ativo
  ansible.builtin.systemd:
    name: slurmdbd
```

```
    state: started
    enabled: true
  when: "'controller' in group_names"
```

Cadastro do cluster e de usuários

O histórico exige que o cluster, as contas e os usuários estejam cadastrados no banco. Essa etapa usa o comando `sacctmgr`:

```
sudo sacctmgr -i add cluster mini-cluster
sudo sacctmgr -i add account pesquisa description="Conta de teste" organiz
sudo sacctmgr -i add user <usuario> account=pesquisa
```

Substitua `mini-cluster` pelo valor de `cluster_name`, e `<usuario>` pelo nome de quem enviará os jobs. A opção `-i` pede confirmação automática.

Esta etapa ainda não é idempotente nesta apostila. Execute-a uma vez, depois confira o resultado:

```
sacctmgr show cluster
sacctmgr show user
```

Verificando o histórico

Envie um job de teste, espere ele terminar, e consulte o histórico:

```
srun hostname
sacct -X --format=JobID,User,Partition,Elapsed,State
```

A consulta deve listar o job com o estado `COMPLETED`. Se o comando responder que o armazenamento de histórico está desativado, o `slurmctld` não está apontado para o slurmdbd. Confira `AccountingStorageType` no `slurm.conf` gerado e o estado de `slurmdbd`:

```
systemctl status slurmdbd
```

> **Referência:** SchedMD. Accounting: Slurm Accounting. slurm.schedmd.com. Arquitetura de contabilidade e papel do slurmdbd. **Referência:** SchedMD. slurmdbd.conf: Slurm Database Daemon configuration file. slurm.schedmd.com. Opções de banco e permissões. **Referência:** SchedMD. sacctmgr: Slurm Accounting Manager. slurm.schedmd.com. Cadastro de clusters, contas e usuários. **Referência:** Ansible Community. community.mysql collection. docs.ansible.com. Módulos `mysql_db` e `mysql_user`. **Referência:** Ansible. Using Ansible Vault to encrypt sensitive data. docs.ansible.com. Criação e uso de valores cifrados. **Referência:** MariaDB Foundation. MariaDB Documentation. mariadb.org. Instalação e administração do servidor.

Resumo da Parte 10

Ao final, você sabe declarar nós virtuais no mesmo host e entende seus limites, inclusive o problema do diretório de spool. Você distribui a chave do munge entre máquinas sem expô-la na saída e sem deixá-la no repositório, e valida a autenticação entre elas. Você também ativa o accounting, com o slurmdbd, o MariaDB e o cadastro de cluster, contas e usuários, e confirma o histórico com `sacct`.

Cada uma dessas funções acrescenta uma dependência. Por isso, a ordem recomendada é validar cada seção antes de seguir para a próxima.

Próxima: Parte 11, Solução de problemas.

Parte 11: Solução de problemas

Objetivo desta parte: organizar o diagnóstico de falhas em um método que funciona para qualquer sintoma: observar, localizar a camada responsável, ler a mensagem de erro e corrigir a fonte.

Pré-requisito: Partes 3 a 10 lidas. Para os exemplos de log, o cluster precisa ter pelo menos um serviço em execução ou falhando.

Esta parte não altera a configuração por si só. Os comandos de diagnóstico são de leitura. As correções indicadas devem ser aplicadas pela fonte, seguindo o fluxo da seção 8.4.

11.1 Tabela de sintomas e causas

Quase todo problema do cluster se encaixa em uma das camadas que a apostila já percorreu: o hardware e o driver, a autenticação, a configuração gerada, os serviços, ou a execução de jobs. A tabela abaixo começa pelo sintoma observado e aponta a camada provável, com a forma de confirmar a causa.

Sintoma	Camada provável	Como confirmar	Correção
`nvidia-smi` não responde, ou o módulo não aparece em `lsmod`	Driver	`lsmod` e `dmesg | grep -i nvidia`	Instalar o driver recomendado, ou tratar o Secure Boot (Parte 2)
Nó aparece como `down` ou `unk` em `sinfo`	Comunicação ou configuração	`scontrol show node <nome>`, campo `Reason`	Ver as linhas seguintes deste quadro
Nó `down` com motivo de hardware	Declaração de recursos	Comparar `CPUTot` e `RealMemory` com `slurmd -C`	Ajustar `cpus` e `real_memory` no inventário, com o valor de `slurmd -C`
`slurmctld` não inicia	Configuração do controlador	`journalctl -u slurmctld -n 50`	Confirmar que `SlurmctldHost` é o hostname real da máquina
`slurmd` não inicia	Autenticação ou configuração	`journalctl -u slurmd -n 50` e `munge -n | unmunge`	Corrigir a chave ou o relógio (linhas seguintes) e rodar o playbook de novo
`Invalid credential` ou erro de munge	Chave ou relógio	`md5sum /etc/munge/munge.key` nas máquinas e	Distribuir a mesma chave e sincronizar o horário com NTP

Sintoma	Camada provável	Como confirmar	Correção
		`date` em cada uma	
Job fica pendente com motivo `Resources`	Capacidade	`squeue` e `scontrol show job <numero>`	Esperar ou cancelar jobs anteriores, ou reduzir o pedido
Job fica pendente com motivo `ReqNodeNotAvail`	Estado do nó	`sinfo -N -l`	Corrigir o nó indisponível antes de reenviar o job
`--gres=gpu:1` é rejeitado na submissão	Declaração de GRES	Conferir `GresTypes` e `Gres=` em `slurm.conf`	Declarar `GresTypes=gpu` antes das linhas de nó
Job roda, mas a GPU não aparece	Dispositivo	`nvidia-smi -L` dentro do job, e `File=` em `gres.conf`	Apontar o caminho correto do dispositivo
Job sem `--gres` enxerga a GPU	Isolamento	Teste negativo da seção 9.3	Usar `File=` em vez de `Count=`, confirmar `ConstrainDevices=yes` e reiniciar `slurmd`
`sacct` informa que o histórico está desativado	Accounting	`grep Accounting /etc/slurm/slurm.conf`	Confirmar `AccountingStorageType` e que `slurmdbd` está ativo
Playbook sempre mostra `changed`	Idempotência	Repetir `--check --diff` e ler o item que muda	Substituir `command` ou `shell` por um módulo específico
Erro `the requested handler could not be found`	Nome de handler	Comparar `notify` com o nome em `handlers/main.yml`	Corrigir o nome para coincidir exatamente
Erro de sintaxe em um template	Jinja2	A mensagem indica o arquivo e a linha	Corrigir a marcação, usando `--check --diff` para validar
`Permission denied` em `/var/spool` ou `/etc/slurm`	Dono ou permissão	`ls -ld <caminho>`	Ajustar `owner`, `group` e `mode` na tarefa de criação
Nó virtual não inicia	Porta ou spool	`journalctl -u slurmd-<nome> -n 50`	Confirmar porta distinta e diretório de spool próprio (Parte 10)

Duas observações sobre a tabela:

- **A ordem importa.** Se o driver não responde, nada acima dele funciona. Se o munge falha, o Slurm falha por consequência. Corrija a camada mais baixa primeiro.
- **A causa aparece antes do sintoma.** Um erro de configuração geralmente se manifesta depois, em um serviço ou em um job. Quando o sintoma não bate com a tabela, volte uma camada e observe a anterior.

  Referência: SchedMD. Slurm Troubleshooting Guide. slurm.schedmd.com. Diagnóstico de nós, jobs e serviços.

11.2 Como ler logs (`journalctl` e logs do Slurm)

Um log registra o que um programa fez e os erros que encontrou. Quando algo falha, a mensagem de erro costuma estar no log, e não na tela. Há duas fontes de log neste cluster.

O journal do systemd

Todos os serviços da apostila (`munge`, `slurmctld`, `slurmd`, `slurmdbd` e os nós virtuais) são gerenciados pelo systemd, que guarda os registros no **journal**. O comando `journalctl` lê esse registro.

Para ver as últimas linhas de um serviço:

```
journalctl -u slurmctld -n 50 --no-pager
```

Para ver o que aconteceu desde um momento específico:

```
journalctl -u slurmd --since "10 minutes ago" --no-pager
```

Para acompanhar um serviço em tempo real, enquanto você reproduz o problema em outro terminal:

```
journalctl -u slurmd -f
```

A opção `-f` continua exibindo as novas linhas até você pressionar `Ctrl+C`.

Os logs do próprio Slurm

O Slurm também pode gravar logs em arquivos próprios, indicados em `slurm.conf` pelas opções `SlurmctldLogFile` e `SlurmdLogFile`. Se essas opções não estiverem definidas, as mensagens vão para o journal, e a seção anterior basta.

Para descobrir onde o Slurm está gravando, consulte a configuração em uso:

```
scontrol show config | grep -i logfile
```

O nível de detalhe das mensagens é controlado pela opção `SlurmctldDebug` e `SlurmdDebug`, e pode ser aumentado temporariamente. Para diagnóstico pontual, o modo de primeiro plano apresentado na Parte 5 (`-D -vvv`) costuma ser mais útil do que mudar o nível do serviço.

Como ler uma mensagem de erro

Uma mensagem de erro tem quatro partes que merecem atenção:

1. **O nome do componente** (`slurmctld`, `slurmd`, `munge`). Ele indica a camada.
2. **O tipo de erro** (`error`, `fatal`, `warning`). `fatal` encerra o processo, e `warning` costuma ser inofensivo, a menos que se repita.
3. **O objeto afetado** (um nó, um arquivo, um job). Ele indica onde procurar.
4. **O texto da causa**, geralmente no fim da linha.

Ao procurar uma mensagem em um log extenso, filtre pelo tipo de erro:

```
journalctl -u slurmctld --no-pager | grep -iE "error|fatal"
```

Leia a primeira mensagem de erro de uma sequência, e não a última. As mensagens seguintes costumam ser consequência da primeira.

Logs do Ansible

Quando o problema está na execução do playbook, a própria saída do Ansible é o log. Para mais detalhes, aumente o nível de verbosidade com `-v`, `-vv` ou `-vvv`:

```
ansible-playbook site.yml -K -vvv
```

A opção `--diff` mostra o conteúdo gerado, e é a forma mais rápida de comparar o que deveria ser com o que está no sistema.

> **Referência:** systemd. journalctl(1). freedesktop.org. Opções de filtro por unidade, tempo e prioridade. **Referência:** SchedMD. slurm.conf: SlurmctldLogFile, SlurmdLogFile, SlurmctldDebug, SlurmdDebug. slurm.schedmd.com. Configuração de logs e níveis de detalhe. **Referência:** Ansible. Debugging Ansible playbooks. docs.ansible.com. Verbosidade e `--diff`.

11.3 Hábitos de depuração

A tabela e os logs resolvem a maioria dos problemas. Os hábitos abaixo resolvem o resto, e evitam que um problema pequeno vire um problema maior.

Mude uma coisa por vez. Quando várias alterações são feitas juntas, não se sabe qual delas causou o efeito, nem qual corrigiu o problema. Aplique uma mudança, rode a verificação, e só então passe para a próxima.

Reproduza antes de corrigir. Um problema que você não consegue provocar de novo não pode ser confirmado como resolvido. Anote o comando exato que falha, para repeti-lo depois da correção.

Corrija a fonte, nunca o resultado. Se um arquivo gerado está errado, a causa está no template ou nas variáveis. Editar o arquivo gerado resolve o sintoma por um instante e o desfaz na execução seguinte (seção 7.4).

Compare o esperado com o real. O `--check --diff` mostra a diferença entre o que o template produziria e o que está no sistema. Essa comparação encontra a maior parte dos desvios de configuração.

Confirme o hardware antes de culpar o software. Muitos erros de nó são declarações de recursos maiores que a máquina. Compare sempre com `slurmd -C`, `nproc` e `free -m`.

Mantenha o histórico. Se o projeto está sob controle de versão, uma mudança que quebrou o cluster pode ser desfeita com um comando. Sem histórico, o caminho de volta depende da memória.

Feche o ciclo. Depois de corrigir, rode a validação completa da Parte 9, e não apenas o comando que falhava. Um conserto pode quebrar outra parte do cluster.

Registre o que aconteceu. Uma nota curta com o sintoma, a causa e a correção evita repetir a investigação. Ela também é útil para quem vier depois.

> **Referência:** Zeller, A. Why Programs Fail: A Guide to Systematic Debugging. Morgan Kaufmann. Método sistemático de depuração, da reprodução à isolação da causa. (Verifique a edição antes de citar.)

Resumo da Parte 11

Ao final, você tem um método de diagnóstico: observar o sintoma, localizar a camada, ler a primeira mensagem de erro nos logs e corrigir a fonte. A tabela de sintomas cobre os casos mais comuns, e os hábitos de depuração cobrem o restante.

Próxima: Parte 12, Conclusão.

Parte 12: Conclusão

Objetivo desta parte: fechar o percurso, mostrar como as camadas construídas se relacionam, indicar as práticas que tornam a automação reutilizável e apontar caminhos para ampliar o cluster.

12.1 O que foi construído

Ao longo da apostila, o cluster foi montado em camadas. Cada camada depende da anterior, e cada uma foi verificada antes de receber a seguinte:

Camada	O que resolve	Partes
Hardware e driver	O sistema reconhece o acelerador e o driver responde	2
Automação	A configuração é descrita em arquivos e aplicada pelo Ansible	3
Pacotes e autenticação	Os componentes estão instalados e se autenticam pelo munge	4
Gerenciador de filas	O Slurm conhece nós, partições e a política de escalonamento	5
Recursos genéricos	As GPUs são reservadas por job e isoladas por cgroups	6
Geração de configuração	Os arquivos do Slurm são produzidos a partir de variáveis	7
Integração	As roles, os handlers e a ordem de execução formam um fluxo único	8
Validação	Os comandos do próprio Slurm confirmam o funcionamento	9
Expansão	O cluster cresce para nós virtuais, várias máquinas e histórico de jobs	10
Diagnóstico	Falhas são localizadas por camada, com logs e um método	11

O resultado é um cluster descrito inteiramente em texto. A topologia está no inventário, o comportamento está nas roles e nos templates, e os valores específicos de cada máquina estão junto da própria máquina. Uma pessoa que não conhece o cluster consegue entender o que ele é lendo esses arquivos, sem depender de memória ou de passos manuais.

Esse é o ponto central da apostila. As ferramentas usadas (Slurm, munge, Ansible, Jinja2, MariaDB) são importantes, mas o método é o que se reutiliza: declarar o estado desejado, gerar a configuração a partir de uma fonte única, verificar antes de aplicar e confirmar depois.

12.2 Boas práticas para reutilizar a automação em outras máquinas

A automação foi escrita para funcionar em uma máquina, mas a mesma estrutura serve para várias. Estas práticas facilitam a reutilização:

Trate o inventário como a fonte da topologia. Cada máquina entra no inventário com o próprio nome, e as variáveis que descrevem o hardware ficam ao lado dela. Adicionar uma máquina é adicionar uma entrada, e nenhum outro arquivo precisa mudar.

Mantenha os valores do cluster separados dos valores das máquinas. Valores como o nome do cluster e a porta padrão ficam em `group_vars/all.yml`. Valores como CPUs, memória e GPUs ficam no host. Essa separação deixa claro o que muda de uma máquina para outra.

Escreva tarefas idempotentes. Uma tarefa que pode ser repetida sem efeito colateral é uma tarefa segura. Prefira módulos específicos a `command` e `shell`, e confira a segunda execução do playbook, que deve mostrar `changed=0`.

Verifique antes de aplicar. A simulação com `--check --diff` mostra o que mudaria. Ela deve ser o passo obrigatório de qualquer alteração, inclusive em máquinas de teste.

Proteja os segredos. Chaves e senhas não entram no repositório em texto puro. Use `no_log` para ocultar a saída das tarefas sensíveis, e um cofre como o Ansible Vault para valores que precisam ser guardados junto do projeto.

Versione a infraestrutura. O histórico de mudanças permite voltar a uma configuração que funcionava e entender por que uma mudança foi feita. Cada alteração relevante deve ser registrada com uma mensagem que explique o motivo.

Documente o ambiente de cada máquina. O arquivo de registro do ambiente (Parte 2) guarda o que é específico de cada máquina: modelo de GPU, versão do driver, estado do Secure Boot. Quem reproduz o cluster sabe em que condições a configuração foi testada.

Não edite o que é gerado. Os arquivos de `/etc/slurm/` são produzidos pelo Ansible. A mudança deve ser feita na fonte, e assim ela sobrevive à execução seguinte e pode ser reproduzida em outra máquina.

Essas práticas não dependem do tamanho do cluster. Um cluster de duas máquinas e um de cinquenta seguem as mesmas regras, e a diferença está apenas no número de entradas do inventário.

> **Referência:** Ansible. Best Practices. docs.ansible.com. Organização de inventários, variáveis, roles e playbooks.
> **Referência:** Morris, K. Infrastructure as Code: Managing Servers in the Cloud. O'Reilly. Princípios de configuração declarativa, idempotência e versionamento. (Verifique a edição antes de citar.)

12.3 Próximos passos

O cluster construído é funcional e reproduzível, mas há áreas que a apostila não aprofundou. Elas podem ser exploradas em seguida, cada uma com o mesmo método apresentado aqui: declarar, gerar, verificar, confirmar.

Políticas de justiça entre usuários (fairshare). Com o accounting ativo (Parte 10), o Slurm pode ajustar a prioridade dos jobs conforme o uso recente de cada usuário. Configurar essa política exige definir contas, pesos e limites, e observar o efeito no histórico. A documentação do Slurm apresenta o plugin `priority/multifactor` como ponto de partida.

Monitoramento contínuo. Os comandos apresentados mostram o estado em um instante. Um monitoramento de séries temporais, com coleta de uso de GPU e de CPU ao longo do tempo, permite identificar ociosidade e gargalos. Ferramentas como o Prometheus, com exportadores para GPU, são o caminho usual.

Armazenamento compartilhado. Quando vários nós precisam dos mesmos dados, um servidor de arquivos em rede (NAS, com NFS ou outra tecnologia) evita cópias. A escolha da solução depende do tamanho do cluster, da rede disponível e do padrão de acesso aos dados. Essa escolha merece a mesma análise de requisitos aplicada aqui.

Alta disponibilidade do controlador. O cluster desta apostila tem um único `slurmctld`. O Slurm suporta um controlador de backup, e a sua configuração exige um diretório de estado compartilhado ou replicado. Essa é uma extensão natural quando o cluster passa a ser usado de forma contínua.

Backup do estado. O diretório `StateSaveLocation` guarda a fila e o estado dos jobs (Parte 5). Copiá-lo periodicamente, junto com o banco de accounting, permite recuperar o cluster depois de uma falha no controlador.

Atualização de versões. O Ubuntu e o Slurm recebem atualizações. Antes de aplicar uma nova versão, vale testá-la em uma máquina separada e confirmar as seções de validação da Parte 9. A verificação com `--check --diff` também indica se uma mudança de pacote altera algum arquivo gerado.

Execução de treinamento. Com o cluster funcionando, o próximo passo natural é executar cargas reais, como o treinamento de modelos de aprendizado de máquina, em jobs enviados pelo `sbatch`. O ambiente de software de cada job pode ser isolado com contêineres, como o Apptainer, o que é um tema próprio.

> **Referência:** SchedMD. Multifactor Priority Plugin. slurm.schedmd.com. Configuração de fairshare e de pesos de prioridade. **Referência:** SchedMD. Quick Start Administrator Guide: Slurm High Availability. slurm.schedmd.com. Controlador de backup e estado compartilhado. **Referência:** Prometheus.

Prometheus Documentation. prometheus.io. Coleta e armazenamento de séries temporais.

Encerramento

A apostila percorreu o caminho completo, do conceito à operação: entender o que é um cluster, verificar o hardware, descrever a configuração em arquivos, aplicá-la de forma repetível, confirmar o funcionamento e encontrar a causa quando algo falha. Os apêndices reúnem o vocabulário, os comandos mais usados e as referências para continuar o estudo.

Apêndice A: Glossário rápido

Os termos estão em ordem alfabética. Cada definição é curta, e a parte da apostila onde o termo é tratado aparece entre parênteses.

Accounting (contabilidade): registro do histórico de jobs, com quem enviou, quando começou, quanto tempo durou e em que estado terminou. No Slurm, é mantido pelo `slurmdbd` (Partes 1 e 10).

Ansible: ferramenta de automação que aplica um estado desejado em máquinas a partir de arquivos YAML, sem exigir programa instalado nelas (Parte 3).

apt: gerenciador de pacotes do Debian e do Ubuntu, usado para instalar e atualizar programas (Parte 2).

cgroups (control groups): recurso do kernel Linux que limita e isola o uso de CPU, memória e dispositivos por grupo de processos (Partes 1 e 6).

Cluster: conjunto de máquinas conectadas em rede que se comportam como um sistema único para quem usa (Parte 1).

Daemon: processo que roda em segundo plano, sem interação direta com o usuário, como o `slurmctld` e o `munge` (Parte 5).

Driver: módulo do kernel que permite ao sistema operacional se comunicar com um dispositivo, como uma GPU (Parte 2).

Fact (fato): informação coletada automaticamente pelo Ansible sobre cada host, como o hostname e a quantidade de memória (Parte 3).

File (`gres.conf`): opção que aponta o arquivo de dispositivo de uma GPU, permitindo que o Slurm isole esse dispositivo por job (Parte 6).

Fonte única da verdade: princípio pelo qual cada valor de configuração existe em um único lugar, e os demais arquivos são gerados a partir dele (Partes 1 e 3).

GPU (Graphics Processing Unit): processador com muitos núcleos, adequado para executar muitas operações iguais em paralelo, como as do treinamento de modelos (Parte 2).

GRES (Generic RESource): recurso genérico com nome livre que o Slurm pode reservar por job. Neste tutorial, o recurso é `gpu` (Parte 6).

Group (grupo de hosts): conjunto nomeado de hosts no inventário, como `controller` e `compute`. As roles usam o grupo para decidir o que cada host executa (Parte 3).

Handler: tarefa que só executa quando uma tarefa a notifica de que algo mudou, geralmente para reiniciar um serviço (Parte 8).

HPC (High Performance Computing): computação de alto desempenho, que reúne máquinas para resolver problemas de cálculo intensivo (Parte 1).

Idempotência: propriedade de uma operação que, repetida, produz o mesmo resultado final. Uma automação idempotente pode ser executada várias vezes sem efeito colateral (Parte 3).

Inventário: arquivo que lista as máquinas gerenciadas pelo Ansible, organizadas em grupos, e guarda as variáveis de cada uma (Parte 3).

IaC (Infrastructure as Code): prática de descrever a configuração de máquinas em arquivos versionados, em vez de configurá-las manualmente (Parte 1).

Jinja2: linguagem de templates que gera texto a partir de variáveis, usando laços e condicionais. O Ansible a usa para produzir os arquivos de configuração (Parte 7).

Job: tarefa enviada a um gerenciador de filas para execução em um ou mais nós (Partes 5 e 9).

journal: registro de eventos mantido pelo systemd, consultado com `journalctl` (Parte 11).

MariaDB: sistema de banco de dados relacional, derivado do MySQL, usado para armazenar o histórico de jobs (Parte 10).

munge: serviço de autenticação que permite aos componentes do Slurm confiar uns nos outros, com base em uma chave compartilhada (Parte 4).

NAS (Network Attached Storage): servidor de arquivos acessado pela rede, que permite a várias máquinas usar os mesmos dados (Parte 1).

NFS (Network File System): protocolo que permite montar uma pasta remota como se fosse local (Parte 1).

Nó: máquina de computação que recebe e executa jobs, descrita no Slurm pela linha `NodeName` (Partes 5 e 9).

Nó virtual (ou simulado): processo `slurmd` adicional, rodando em um host real, que se apresenta ao Slurm como um nó separado. Serve para testar o escalonador, não para medir desempenho (Parte 10).

Partição: grupo de nós com regras comuns de acesso, escolhido pelo usuário ao enviar um job (Partes 5 e 9).

Playbook: arquivo do Ansible que lista as roles a aplicar, em quais hosts e com quais privilégios (Parte 3).

Role: pasta com estrutura padronizada de tarefas, handlers, templates e variáveis de um assunto, reutilizável em vários playbooks (Partes 3 e 8).

Slurm (Slurm Workload Manager): gerenciador de recursos e escalonador de jobs de código aberto, usado em clusters de computação (Parte 5).

slurmctld: daemon controlador do Slurm, que mantém a fila e decide as alocações (Parte 5).

slurmd: daemon de cada nó de computação, que executa os jobs e informa o estado do nó (Parte 5).

slurmdbd: daemon que grava o histórico de jobs em um banco de dados (Partes 5 e 10).

Spool: diretório de trabalho temporário de um serviço. No `slurmd`, guarda arquivos enquanto um job executa (Partes 4 e 10).

sudo: comando que executa uma ação com privilégio de administrador (Parte 2).

systemd: gerenciador de serviços do Linux, que inicia, para e monitora daemons (Partes 4 e 11).

Template: arquivo com espaços reservados para valores, que o Ansible transforma em arquivo final (Parte 7).

Variável de host: valor que pertence a uma máquina específica, definido no inventário (Parte 3).

YAML: formato de texto para dados estruturados, em que a indentação com espaços define a hierarquia (Parte 1).

Apêndice B: Comandos de referência

Os comandos estão agrupados pela tarefa que realizam. Os que alteram o sistema são marcados com **(altera)**. Os demais apenas leem o estado.

Todos os comandos do Ansible são executados a partir da raiz do projeto.

Verificar o hardware

```
nvidia-smi -L                          # lista as GPUs NVIDIA com o modelo co
lspci | grep -iE 'vga|3d|display'      # dispositivos gráficos no barramento
lsmod | grep -i nvidia                 # módulo do driver carregado
hostname -s                            # nome curto da máquina
nproc                                  # número de CPUs disponíveis
free -m                                # memória total e em uso
slurmd -C                              # configuração de hardware vista pelo
```

Verificar o sistema

```
cat /etc/os-release                    # distribuição e versão
mokutil --sb-state                     # estado do Secure Boot
apt policy <pacote>                    # versão disponível e instalada de um
stat -fc %T /sys/fs/cgroup             # versão do cgroup (cgroup2fs indica a
```

Ansible: inventário e variáveis

```
ansible-inventory --graph              # grupos e hosts, como o Ansible os in
ansible-inventory --host <hostname> # variáveis de um host específico
ansible all -m ansible.builtin.debug -a "var=cluster_name"   # valor de um
ansible-config dump --only-changed  # opções do ansible.cfg em uso
ansible-doc <modulo>                   # documentação de um módulo
```

Ansible: execução

```
ansible-playbook site.yml --syntax-check       # valida a sintaxe
ansible-playbook site.yml --list-tasks         # lista as tarefas
ansible-playbook site.yml --check --diff -K    # simula e mostra as difere
ansible-playbook site.yml -K                   # (altera) aplica o playboo
ansible-playbook site.yml -K -vvv              # (altera) aplica com saída
```

A opção `-K` pede a senha do `sudo` antes de começar.

Ansible: comandos avulsos

```
ansible all -m ansible.builtin.ping                                  # testa a
ansible all -m ansible.builtin.setup -a "filter=ansible_hostname"   # fato
ansible all -b -m ansible.builtin.command -a "md5sum /etc/munge/munge.key"
```

Pacotes e serviços

```
sudo apt update                                  # (altera) atualiza a lis
sudo apt install -y <pacote>                     # (altera) instala um pac
systemctl status <servico>                       # estado de um serviço
sudo systemctl restart <servico>                 # (altera) reinicia um se
systemctl list-units --type=service | grep slurm # serviços do Slurm
```

Autenticação (munge)

```
munge -n | unmunge                              # testa a autenticação loc
munge -n | ssh <host> unmunge                   # testa a autenticação ent
sudo ls -l /etc/munge/munge.key                 # dono e permissões da cha
```

Estado do cluster (Slurm)

```
sinfo                                # partições e estados dos nós
sinfo -N -l                          # um nó por linha, com detalhes
scontrol show node <nome>            # recursos e motivo de estado de um nó
scontrol show job <numero>           # detalhes de um job
squeue                               # fila de jobs
```

Enviar e controlar jobs

```
srun hostname                        # executa um comando de imediato
srun --gres=gpu:1 nvidia-smi -L      # executa com uma GPU alocada
sbatch <script>.sh                   # envia um script para a fila
sbatch --gres=gpu:1 --wrap="<comando>"   # envia um comando direto
scancel <numero>                     # cancela um job
sudo scontrol update nodename=<nome> state=resume   # (altera) devolve um
```

Histórico (accounting)

```
sacct -X --format=JobID,User,Partition,Elapsed,State   # jobs concluídos
sacctmgr show cluster                                  # clusters cadastra
sacctmgr show user                                     # usuários cadastra
sudo sacctmgr -i add cluster <nome>                    # (altera) cadastra
```

Logs e diagnóstico

```
journalctl -u slurmctld -n 50 --no-pager          # últimas linhas de um s
journalctl -u slurmd -f                           # acompanha um serviço e
journalctl -u slurmctld --no-pager | grep -iE "error|fatal"   # filtra err
sudo slurmd -D -vvv                               # executa o slurmd em pr
scontrol show config | grep -i logfile            # onde o Slurm grava log
```

Versionamento

```
git status                          # mudanças pendentes
git diff                            # o que mudou desde o último commit
git add <arquivos>                  # (altera o histórico local) prepara a
git commit -m "<mensagem>"          # (altera o histórico local) registra
git log --oneline                   # histórico de versões
```

Apêndice C: Referências para aprofundamento

As referências estão agrupadas por tipo. Todas são fontes públicas: documentação oficial dos projetos, artigos científicos publicados e livros de editora. Para cada assunto, a documentação oficial é a fonte de consulta mais atualizada, porque acompanha as versões. Os livros e artigos complementam a compreensão dos conceitos.

Documentação oficial

Slurm (SchedMD). Documentação do gerenciador de filas, incluindo o guia de administração, as páginas de configuração e as páginas de comandos. Disponível em slurm.schedmd.com.

- Slurm Quick Start Administrator Guide. slurm.schedmd.com/quickstart_admin.html
- slurm.conf: Slurm configuration file. slurm.schedmd.com/slurm.conf.html
- gres.conf: Generic Resource configuration file. slurm.schedmd.com/gres.conf.html
- cgroup.conf: Slurm cgroup support configuration file. slurm.schedmd.com/cgroup.conf.html
- slurmdbd.conf: Slurm Database Daemon configuration file. slurm.schedmd.com
- Generic Resource (GRES) Scheduling. slurm.schedmd.com/gres.html
- sbatch, srun, scancel, sinfo, squeue, scontrol e sacctmgr. slurm.schedmd.com
- Multifactor Priority Plugin. slurm.schedmd.com

Ansible. Documentação do Ansible, com guias, referência de módulos e de variáveis. Disponível em docs.ansible.com.

- How to build your inventory
- Using variables e Magic variables, and how to access information about other hosts
- Playbooks e Using roles
- Handlers: running operations on change
- Templating (Jinja2)
- Validating tasks: check mode and diff mode
- Using Ansible Vault to encrypt sensitive data
- Best Practices

Ansible Community: community.mysql. Módulos para bancos e usuários MySQL e MariaDB, em docs.ansible.com.

Jinja2 (Pallets). Documentação da linguagem de templates, incluindo variáveis, laços, condicionais e filtros. Disponível em jinja.palletsprojects.com.

munge. Documentação do serviço de autenticação, em dun.github.io/munge.

Linux kernel. Documentação da versão unificada dos cgroups, em docs.kernel.org/admin-guide/cgroup-v2.html.

systemd. Documentação das unidades de serviço e do registro de eventos, em freedesktop.org (páginas `systemd.service(5)` e `journalctl(1)`).

NVIDIA. Guia do usuário do `nvidia-smi`, em docs.nvidia.com.

Ubuntu. Consulta de pacotes por distribuição, em packages.ubuntu.com.

Multipass (Canonical). Documentação de criação de máquinas virtuais e uso de `cloud-init`, em multipass.run.

MariaDB Foundation. Documentação do servidor de banco de dados, em mariadb.org.

Prometheus. Documentação do sistema de coleta de séries temporais, em prometheus.io.

Git. Documentação e o livro Pro Git, de Chacon e Straub, disponível gratuitamente em git-scm.com.

Artigos científicos

Yoo, A. B.; Jette, M. A.; Grondona, M. SLURM: Simple Linux Utility for Resource Management. In: Job Scheduling Strategies for Parallel Processing (JSSPP), Springer, 2003. Descreve a arquitetura original do Slurm.

Feitelson, D. G.; Rudolph, L.; Schwiegelshohn, U.; Sevcik, K. C.; Wong, P. Theory and Practice in Parallel Job Scheduling. In: Job Scheduling Strategies for Parallel Processing (JSSPP), Springer, 1997. Trata a teoria das políticas de escalonamento, como FIFO e backfill.

Livros

Ansible e automação.

- Geerling, J. Ansible for DevOps. Leanpub.
- Morris, K. Infrastructure as Code: Managing Servers in the Cloud. O'Reilly.

Linux e sistemas.

- Kerrisk, M. The Linux Programming Interface. No Starch Press.
- Shotts, W. The Linux Command Line. No Starch Press.
- Gregg, B. Systems Performance: Enterprise and the Cloud. Addison-Wesley.

Depuração.

- Zeller, A. Why Programs Fail: A Guide to Systematic Debugging. Morgan Kaufmann.

Como usar esta lista

Para um assunto específico, comece pela documentação oficial da ferramenta, na versão que você instalou. Quando o conceito precisar de mais profundidade, recorra ao livro ou ao artigo indicado na seção de cada parte. Os artigos científicos são úteis para entender a origem das ideias, como o escalonamento por backfill e a arquitetura do Slurm.